Models and Tools for Cyber-Physical Systems

UPPAAL

Modelling, Specification and Verification

Instructors: Suman Sourav, Jonathan Julian Huerta y Munive
{sumansourav, jjhymuni}@cs.aau.dk

Slides courtesy of Kim G. Larsen

Agenda

- UPPAAL
  - History and Demos
- Train Gate Example
  - Modelling Formalism
    (Networks of Extended Timed Automata)
  - Specification Formalism
- UPPAAL Engine
- UPPAAL Options

Quantiative aspects
(timing, energy, bandwitdt, memory,..)
crucial for several embedded systems

Main Readings (+Prin. of CPS chapter 7)

Chapter 1
A First Introduction to Uppaal

Frits Vaandrager

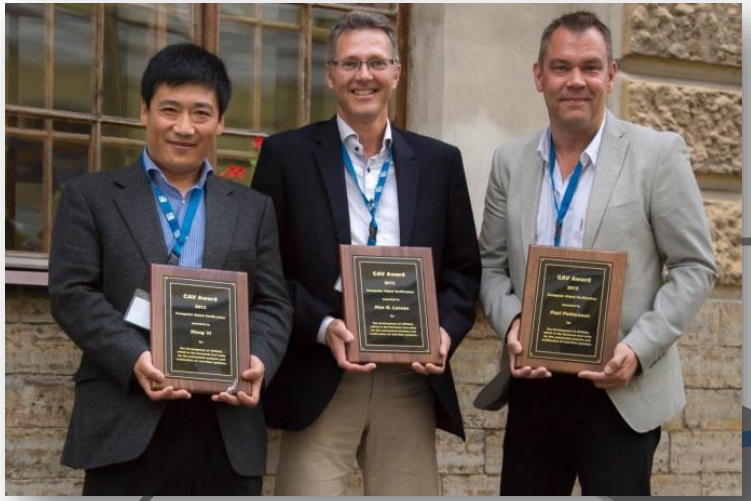
Abstract This chapter provides a first introduction to the use of the model checking tool Uppaal. Uppaal is an integrated tool environment that allows users to model the behavior of systems in terms of states and transitions between states, and to simulate and analyze the resulting models. Uppaal can also handle real-time issues, that is, the timing of transitions. Using an example of a jobshop, we explain in a step by step manner how one can make a simple Uppaal model, simulate its behavior and analyze its properties. This introduction is targeted at a broad audience, ranging from high school students to software engineers and researchers. We only require elementary knowledge of programming and mathematics. Although a rich theory of model checking has been developed over the last decades, which includes both clever algorithms and deep mathematics, this introduction focuses entirely on the

Chapter 1
More Features in UPPAAL

Alexandre David, Kim G. Larsen

Abstract Following the introduction to the model checking tool UPPAAL of the previous chapter, this chapter presents a number of additional modeling and verification features offered by the tool. These features include in particular a C-like imperative language with user-defined types and functions, allowing for readable and compact models with reusable updates of discrete variables. Using an example of a Train Gate, we demonstrate the use(fulness) of these features. Also, the chapter presents the full query language of UPPAAL covering both safety, liveness and time-bounded liveness properties, again illustrated using the Train Gate example. Finally, directions are given on modelling choices and use of verification options that may improve time- and/or space-performance of the UPPAAL verifier

Origin of UPPAAL



TAU
CCS & Modal Transition Systems
Refinements
Modal Mu-Calculus
Explicit State Representation
Prolog

1995

UPPAAL
Timed Automata
TCTL
Zones
C++ & Java

2007

EPSILON
TCCS
Timed Refinements
Timed Mu-Calculus
Regions
Prolog<

2013

2016

UP4ALL

2018

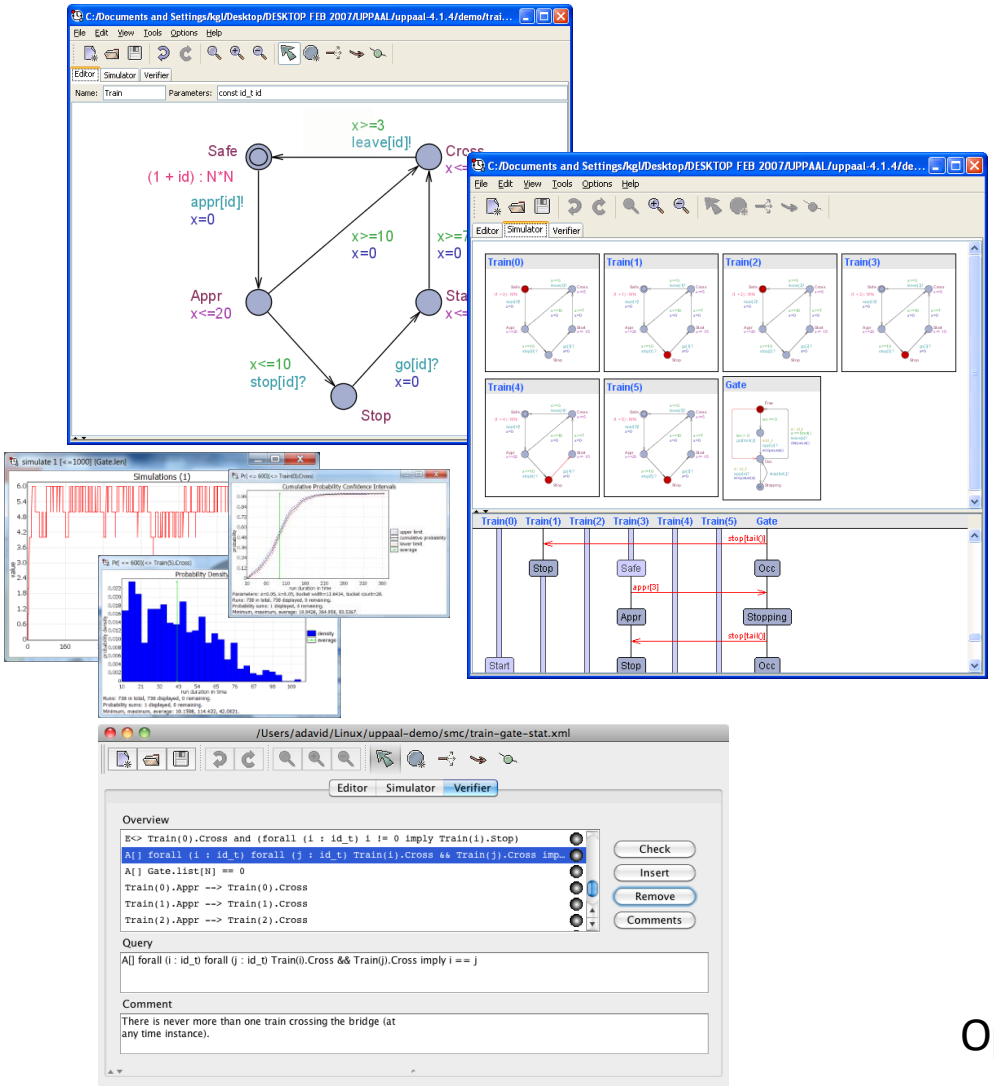
CAV Award

Grundfos Prize

ATS VeriAll

AALBORG UNIVERSITET

UPPAAL Tool Suit



Verification	CLASSIC	1995
Optimization	CORA	2001
Synthesis	TIGA	2005
Component	ECDAR	2010
Testing	TRON	2004
Performance Analysis	SMC	2011
Optimal Synthesis	STRATEGO	2014

Contributors

@UPPsala

- Wang Yi
- Paul Pettersson
- John Håkansson
- Anders Hessel
- Pavel Krcal
- Leonid Mokrushin
- Shi Xiaochun

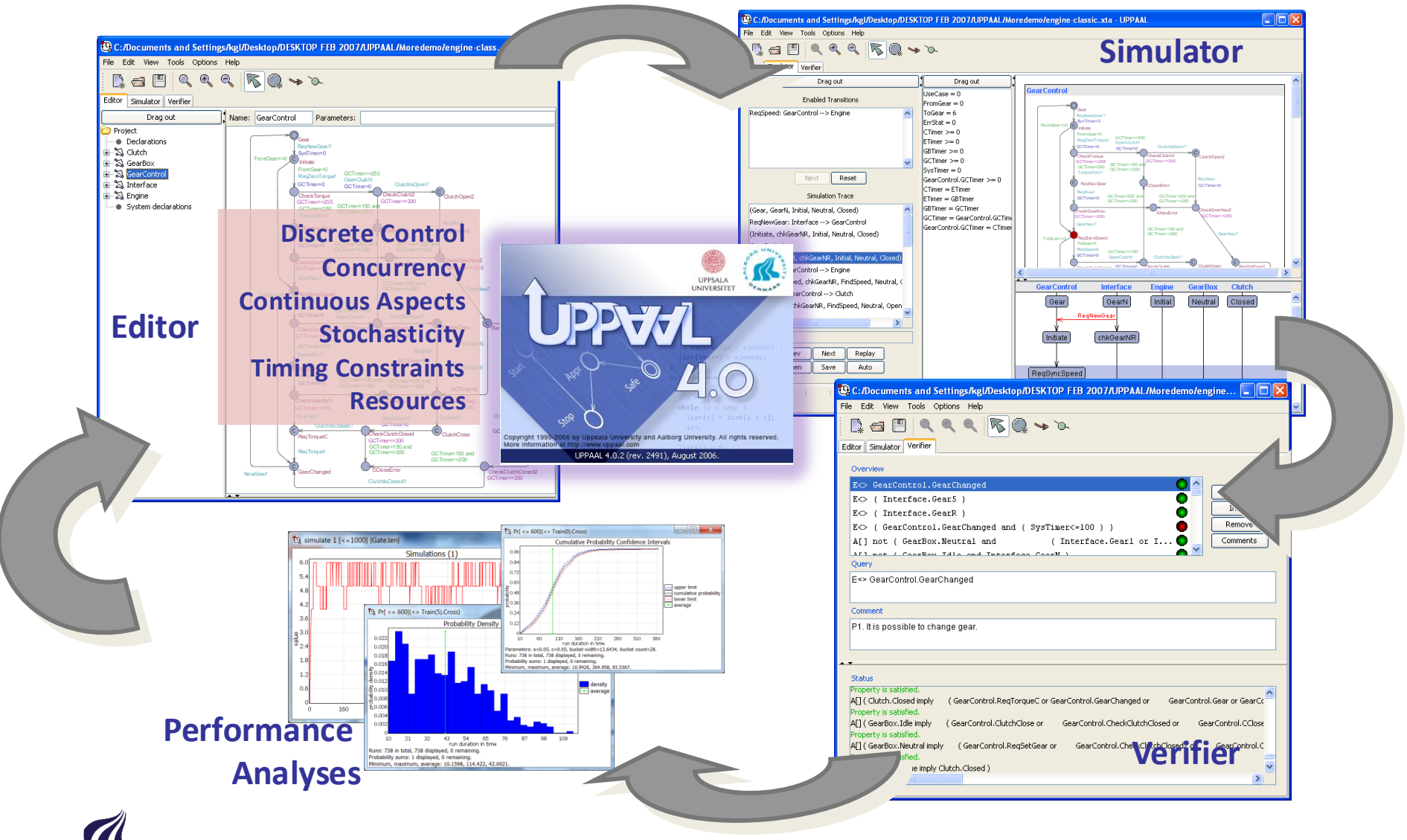
@AALborg

- Marius Mikucionis
- Peter Gjøl Jensen
- Kenneth Y Jørgensen
- Danny Poulsen
- Marco Muniz
- Ulrik Nyman
- Arne Skou
- Brian Nielsen
- Alexandre David
- Gerd Behrman
- Jacob Illum Rasmussen
- Kim G Larsen

@Elsewhere

- Emmanuel Fleury, Didier Lime, Johan Bengtsson, Fredrik Larsson, Kåre J Kristoffersen, Tobias Amnell, Thomas Hune, Oliver Möller, Elena Fersman, Carsten Weise, David Griffioen, Ansgar Fehnker, Frits Vandraager, Theo Ruys, Pedro D'Argenio, J-P Katoen, Jan Tretmans, Judi Romijn, Ed Brinksma, Martijn Hendriks, Klaus Havelund, Franck Cassez, Magnus Lindahl, Francois Laroussinie, Patricia Bouyer, Augusto Burgueno, H. Bowmann, D. Latella, M. Massink, G. Faconti, Kristina Lundqvist, Lars Asplund, Justin Pearson...

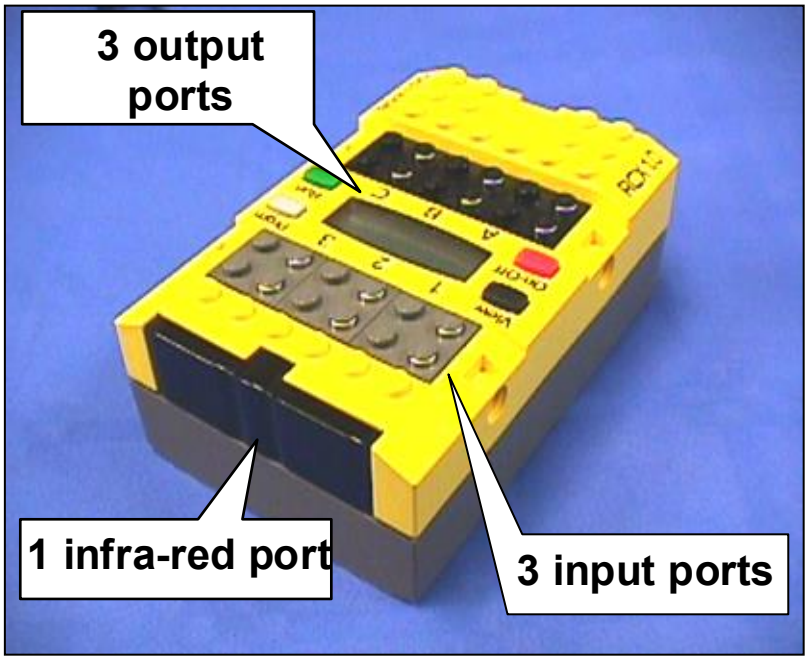
UPPAAL Model Checker



Brick Sorting

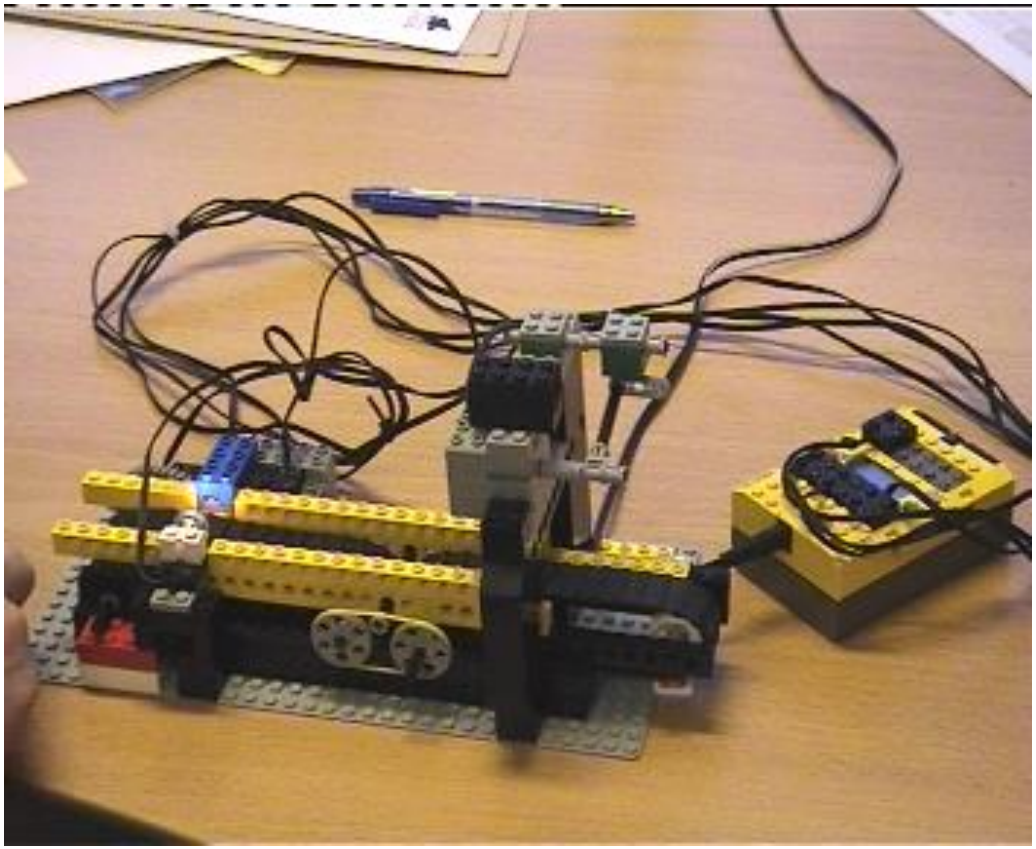
LEGO Mindstorms/RCX

- Sensors: temperature, light, rotation, pressure.
- Actuators: motors, lamps,
- Virtual machine:
  - 10 tasks, 4 timers, 16 integers.
- Several Programming Languages:
  - NotQuiteC, Mindstorm, Robotics, legOS, etc.



A Real Real Timed System

The Plant
Conveyor Belt
&
Bricks

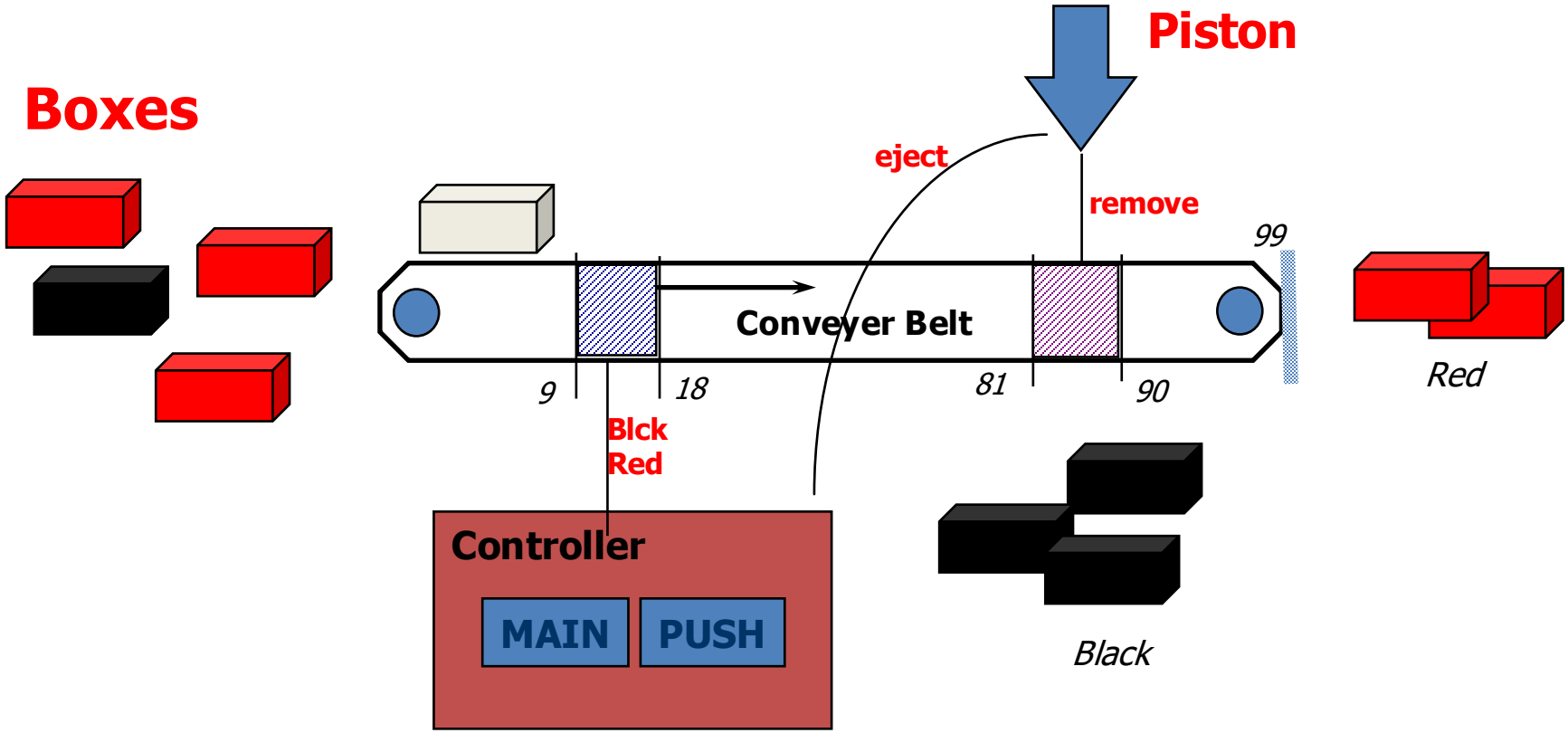


Controller Program
LEGO MINDSTORM

First UPPAAL model

Sorting of Lego Boxes

Ken Tindell



Exercise: Design **Controller** so that **black** boxes are being pushed out

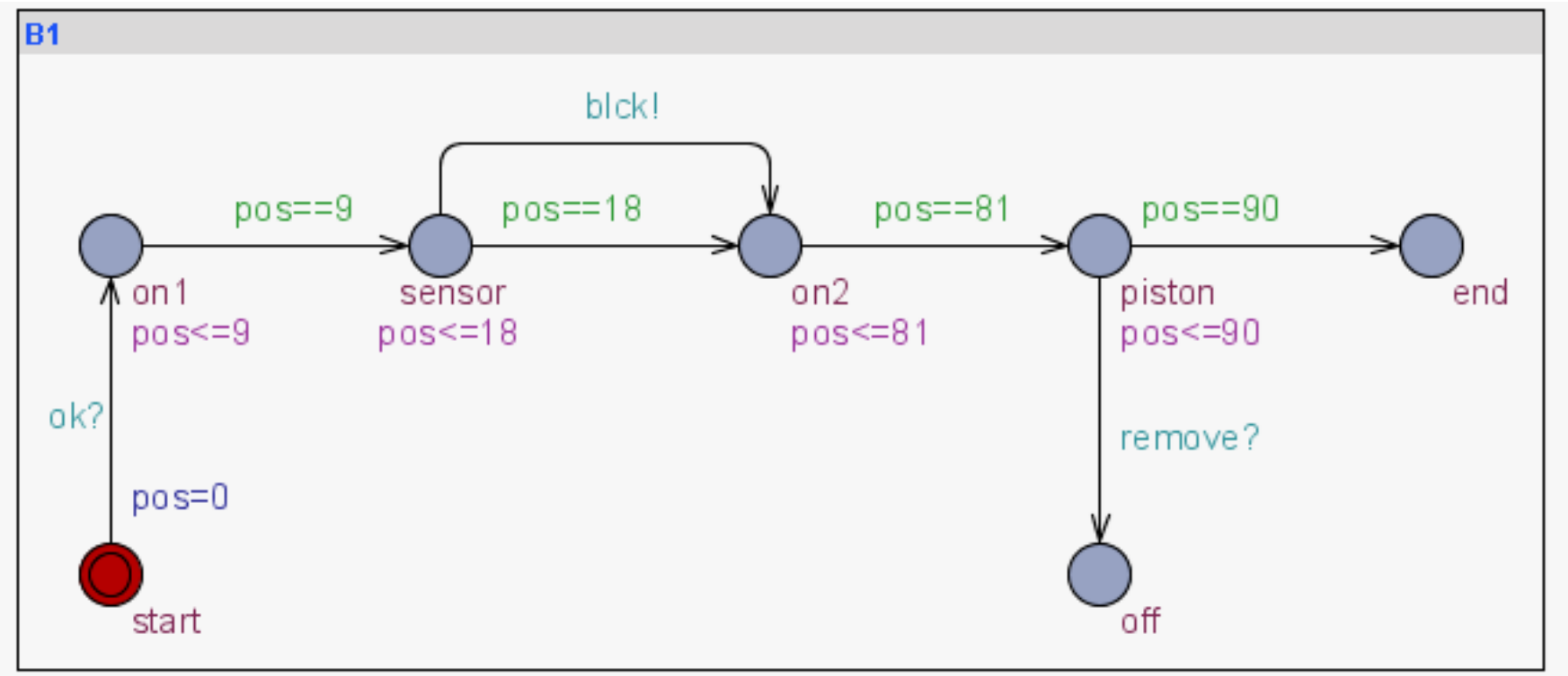
NQC programs

```
int active;
int DELAY;
int LIGHT_LEVEL;
```

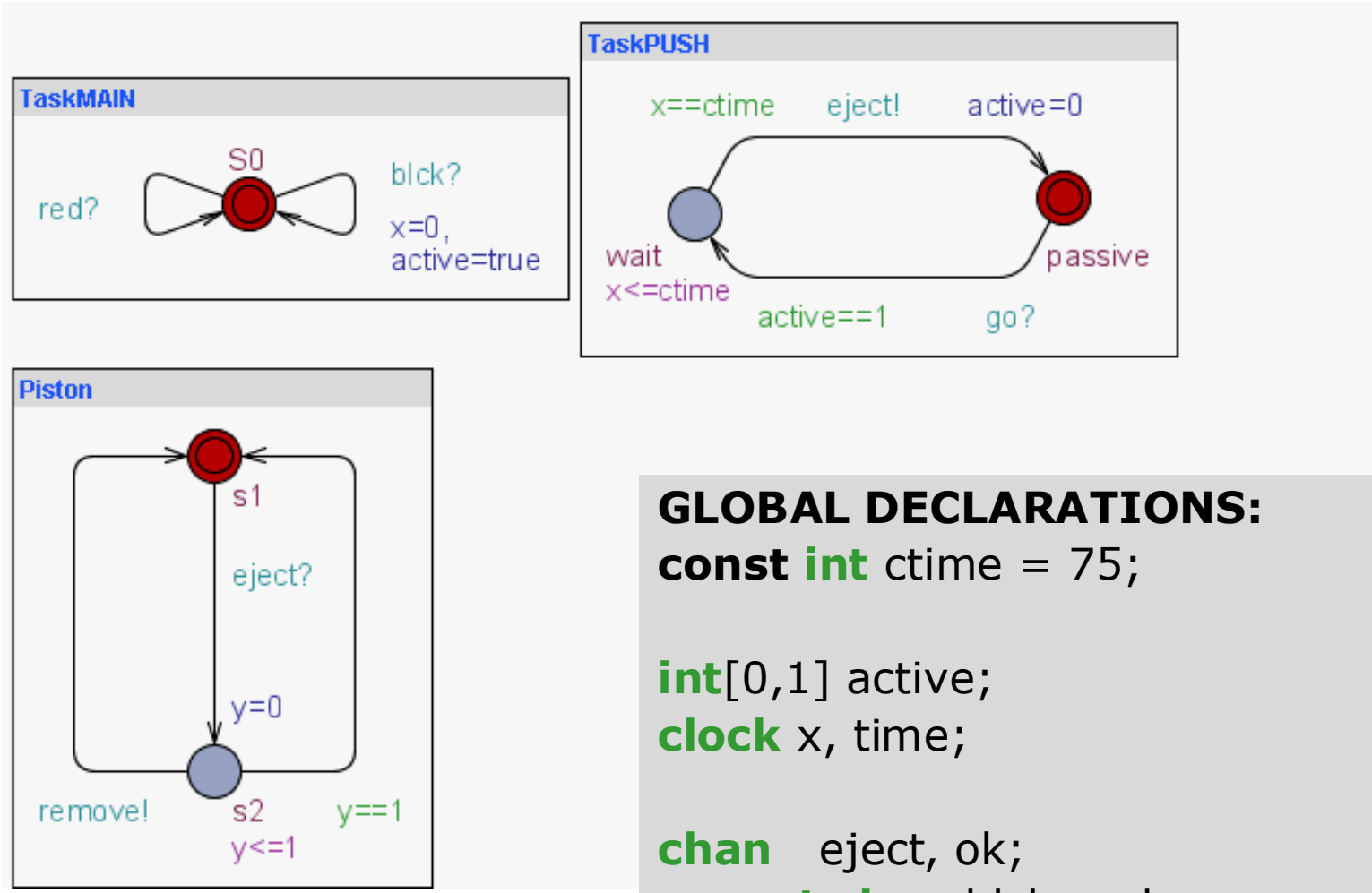
```
task MAIN{
 DELAY=75;
 LIGHT_LEVEL=35;
 active=0;
 Sensor(IN_1, IN_LIGHT);
 Fwd(OUT_A,1);
 Display(1);

 start PUSH;

 while(true){

wait(IN_1<=LIGHT_LEVEL);
   ClearTimer(1);
   active=1;
   PlaySound(1);

wait(IN_1>LIGHT_LEVEL);
  }
}
```

```
task PUSH{
  while(true){
    wait(Timer(1)>DELAY && active==1);
    active=0;
    Rev(OUT_C,1);
    Sleep(8);
    Fwd(OUT_C,1);
    Sleep(12);
    Off(OUT_C);
  }
}
```

A Black Brick



Control Tasks & Piston

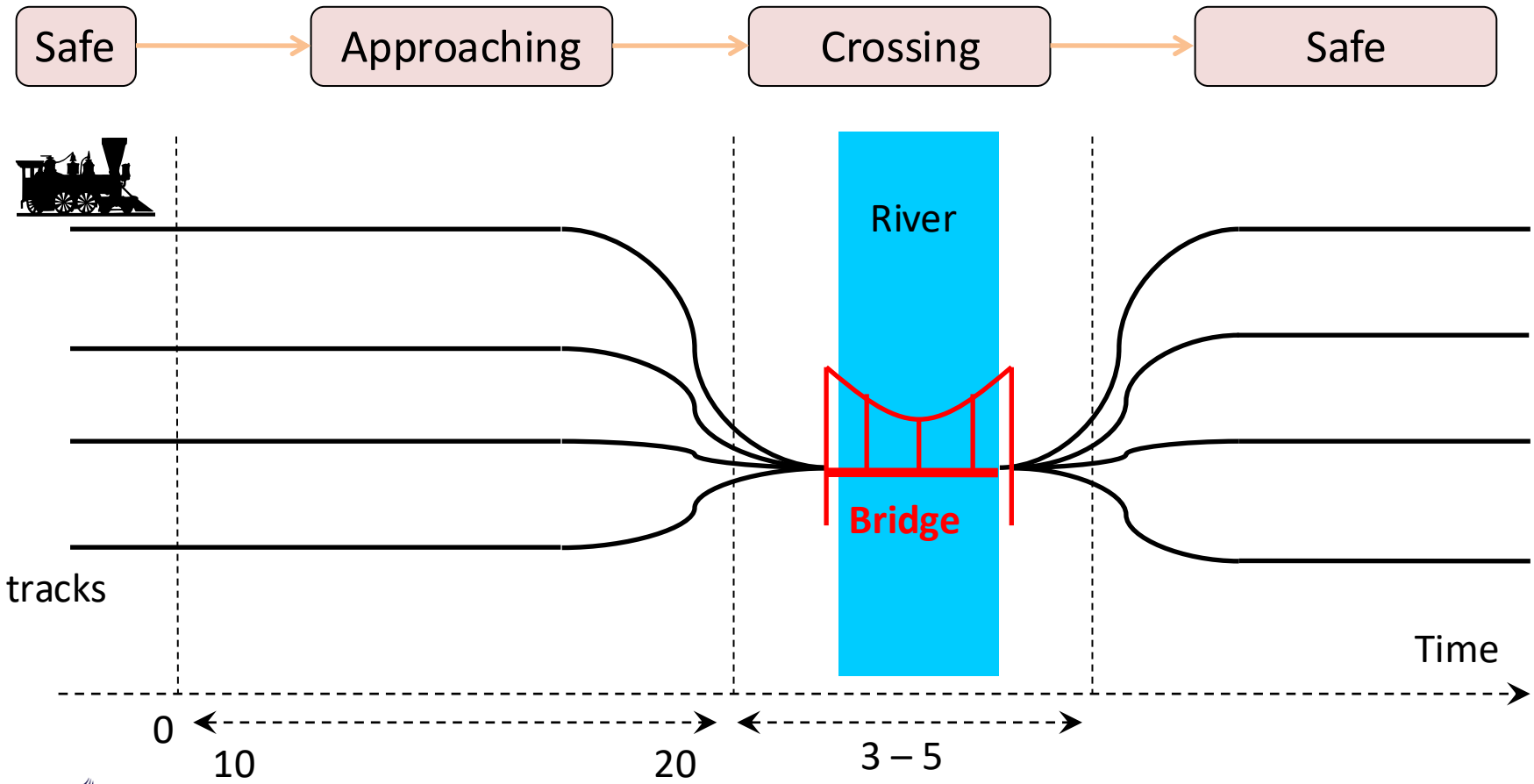


GLOBAL DECLARATIONS:

```
const int ctime = 75;

int[0,1] active;
clock x, time;

chan    eject, ok;
urgent chan blck, red, remove, go;
```

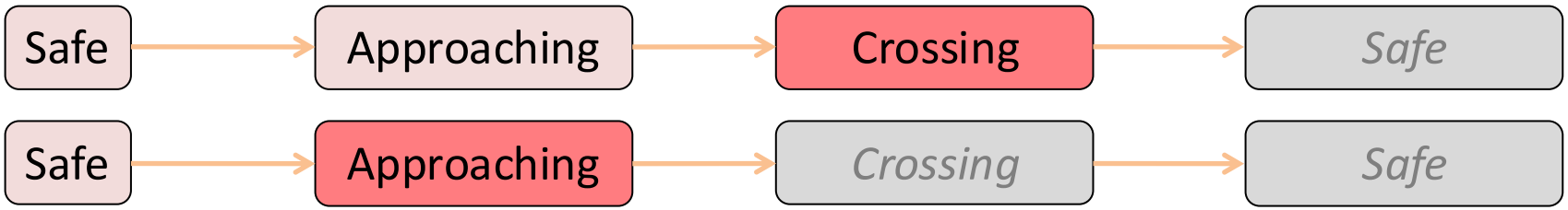
Train Crossing

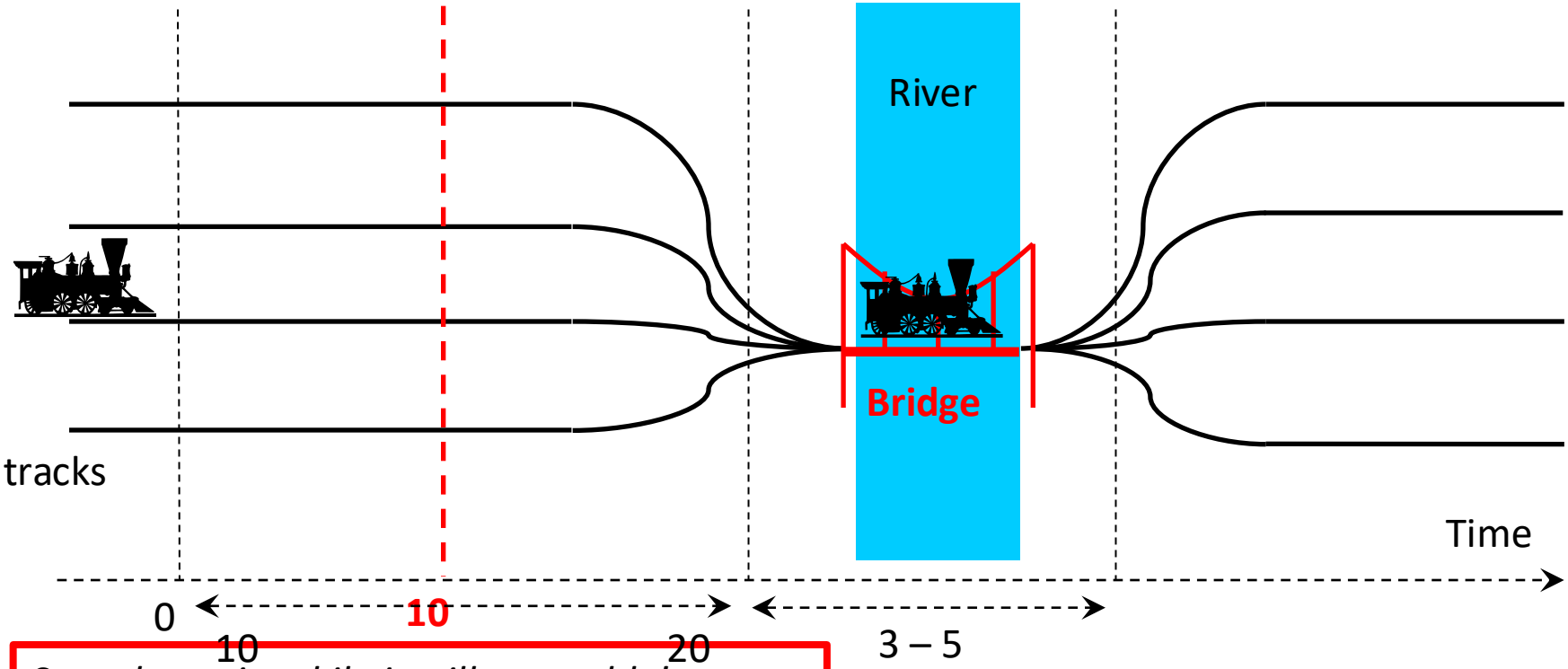
Train Crossing



AALBORG UNIVERSITET

Train Crossing

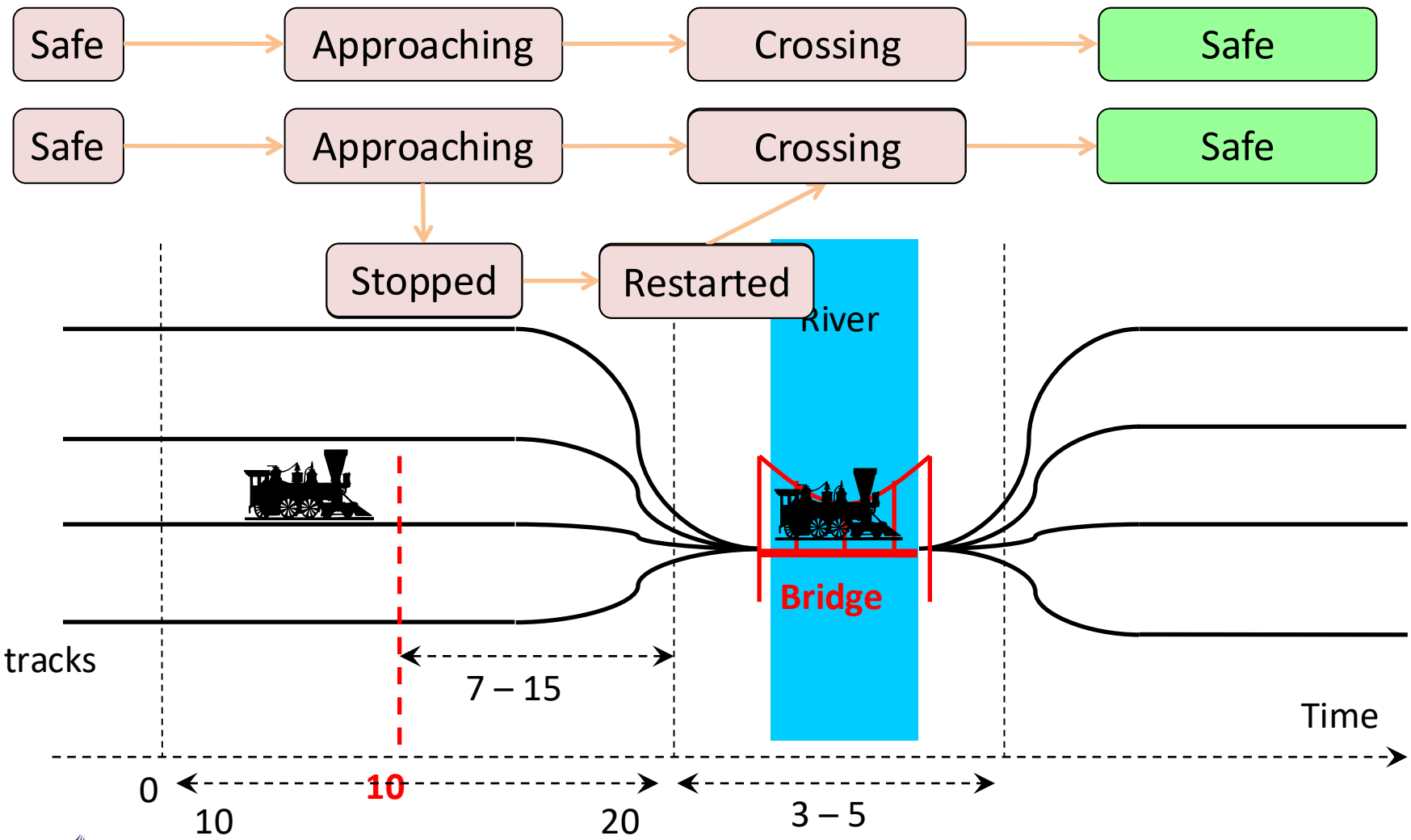




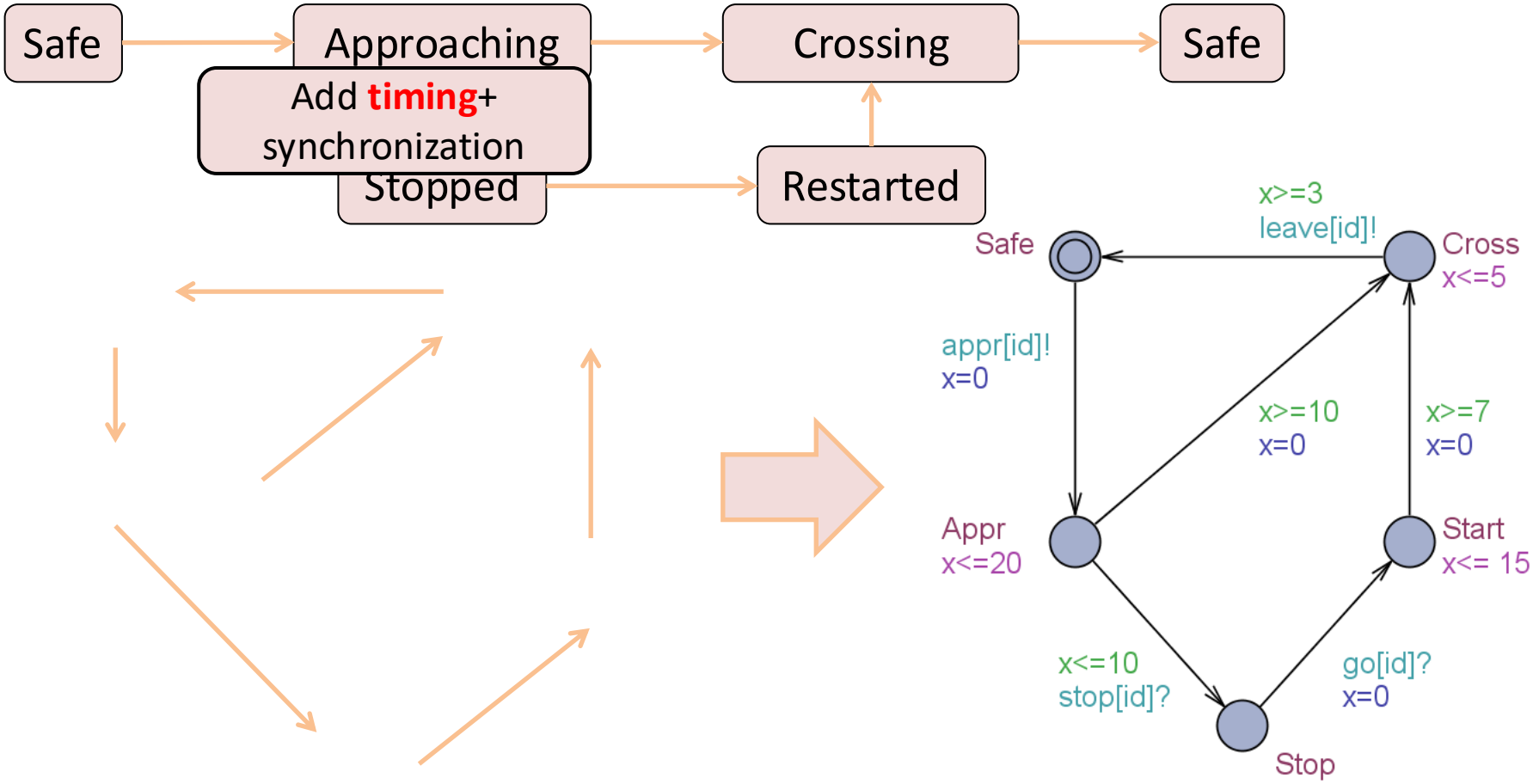
Stop the train while it still stoppable!

Miro board link: https://tinyurl.com/MTCPS-train-gate

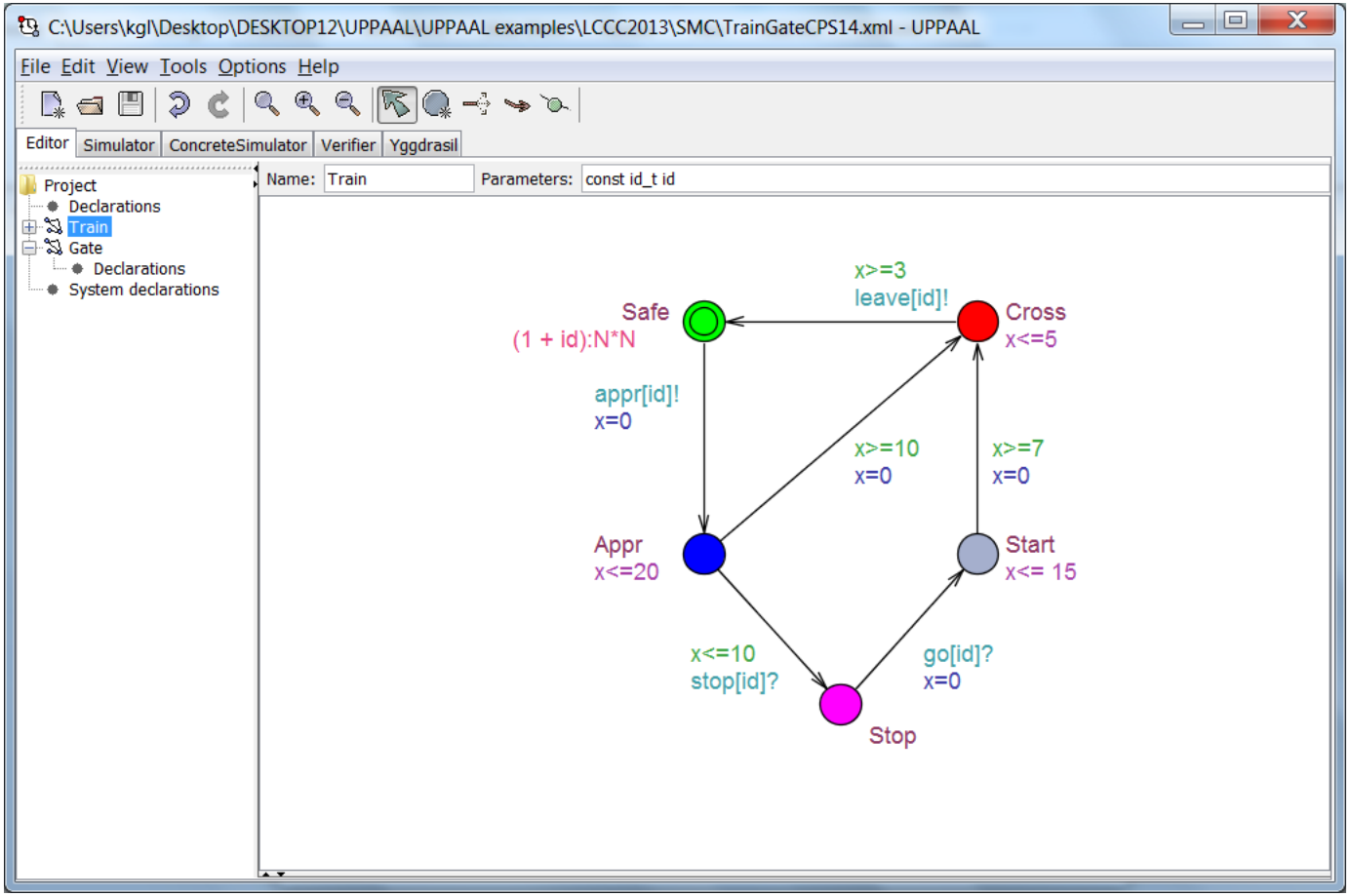
Train Crossing



Train Crossing



Demo



AALBORG UNIVERSITET

Logical Specifications

- Validation Properties
  - Possibly: E<> *P*
- Safety Properties
  - Invariant: A[] *P*
  - Pos. Inv.: E[] *P*
- Liveness Properties
  - Eventually: A<> *P*
  - Leadsto: P → *Q*
- Bounded Liveness
  - Leads to within: $P \rightarrow_{\leq t} Q$

The expressions *P* and *Q* must be type safe, side effect free, and evaluate to a boolean.

Only references to integer variables, constants, clocks, and locations are allowed (and arrays of these).

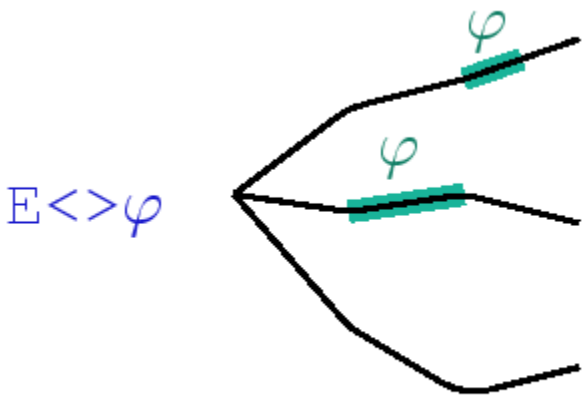
Logical Specifications

- Validation Properties
  - Possibly: E<> *P*

- Safety Properties
  - Invariant: A[] *P*
  - Pos. Inv.:E[] *P*

- Liveness Properties
  - Eventually: A<> *P*
  - Leadsto: P → *Q*

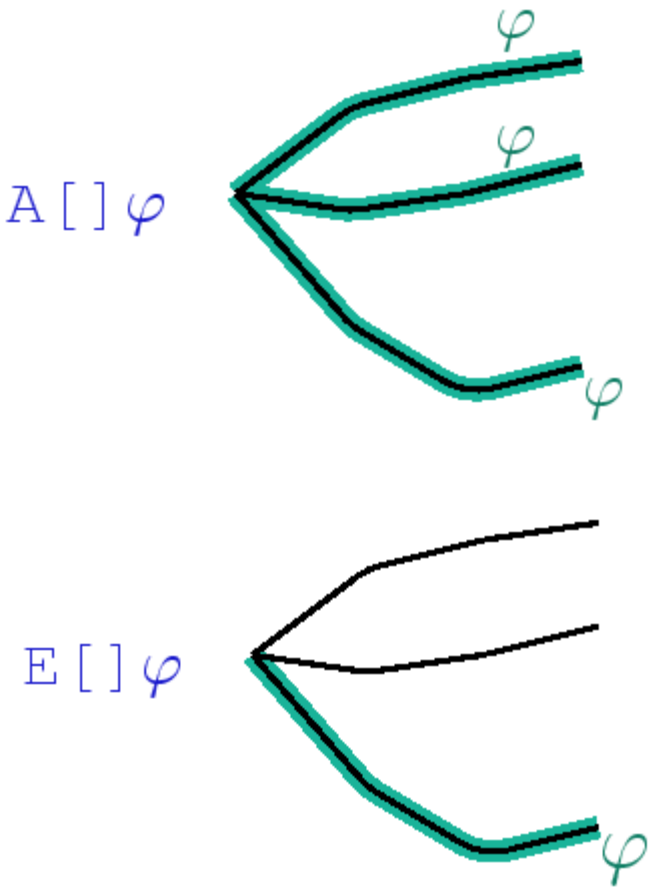
- Bounded Liveness
  - Leads to within: $P \rightarrow_{\leq t} Q$



AALBORG UNIVERSITET

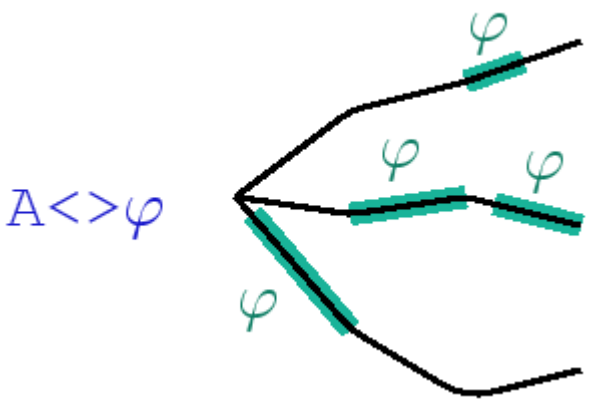
Logical Specifications

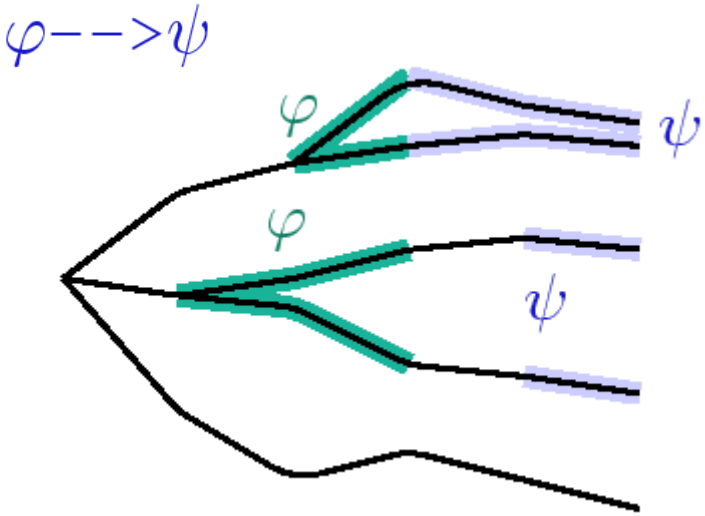
- Validation Properties
  - Possibly: E<> *P*
- Safety Properties
  - Invariant: A[] *P*
  - Pos. Inv.: E[] *P*
- Liveness Properties
  - Eventually: A<> *P*
  - Leadsto: P → *Q*
- Bounded Liveness
  - Leads to within: $P \rightarrow_{\leq t} Q$



Logical Specifications

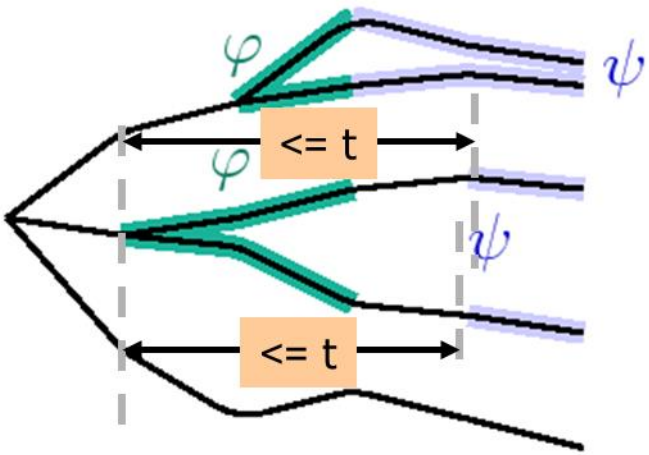
- Validation Properties
  - Possibly: E<> *P*
- Safety Properties
  - Invariant: A[] *P*
  - Pos. Inv.:E[] *P*
- Liveness Properties
  - Eventually: A<> *P*
  - Leadsto: P → *Q*
- Bounded Liveness
  - Leads to within: $P \rightarrow_{\leq t} Q$



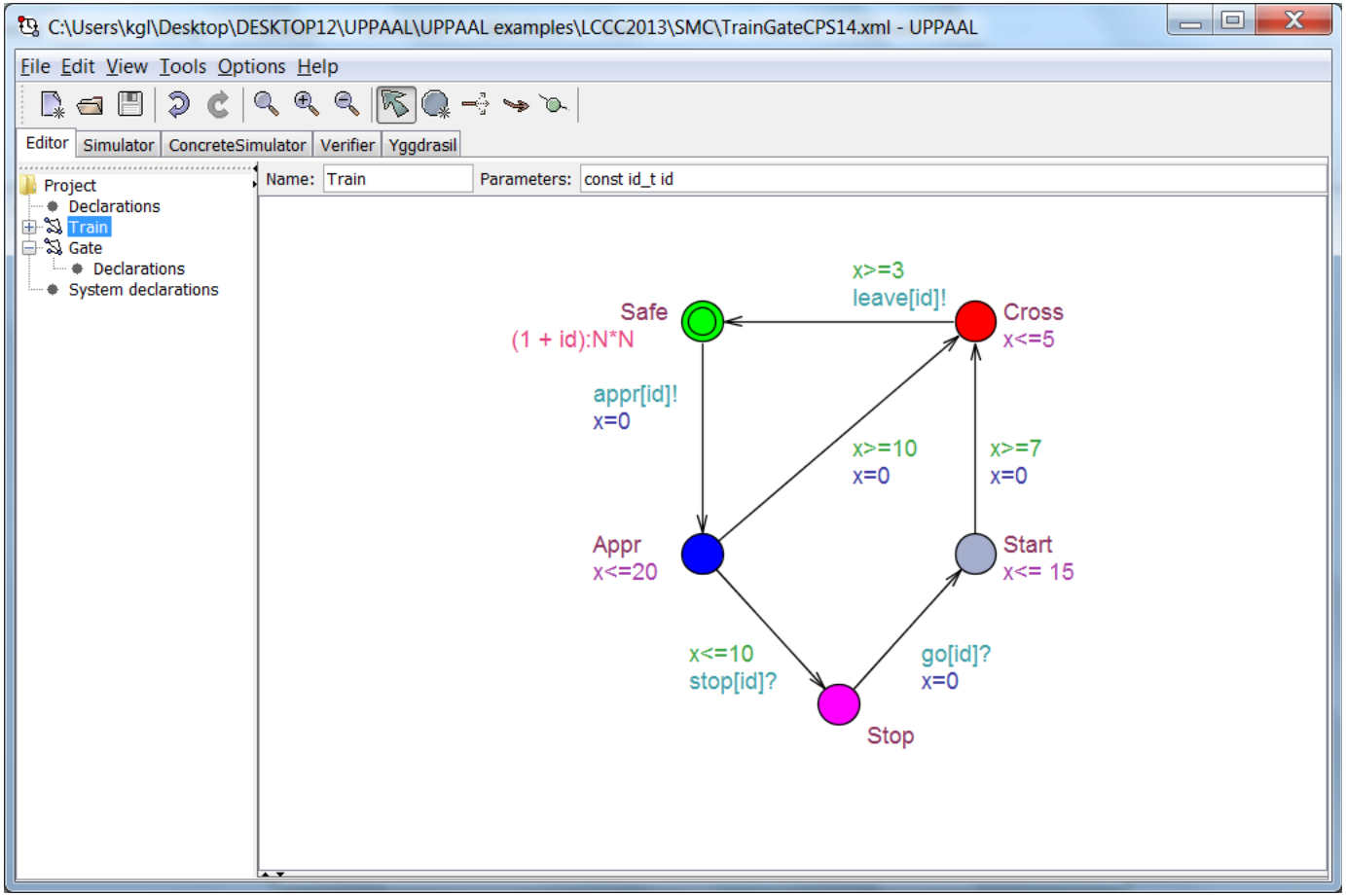


Logical Specifications

- Validation Properties
  - Possibly: E<> *P*
- Safety Properties
  - Invariant: A[] *P*
  - Pos. Inv.:E[] *P*
- Liveness Properties
  - Eventually: A<> *P*
  - Leadsto: P → *Q*
- Bounded Liveness
  - Leads to within: $P \rightarrow_{\leq t} Q$



Demo



AALBORG UNIVERSITET

Editor

C:\Users\kgl\Desktop\DESKTOP12\UPPAAL\UPPAAL examples\LCCC2013\SMC\TrainGateCPS1...

File Edit View Tools Options Help

C:\Users\kgl\Desktop\DESKTOP12\UPPAAL\UPPAAL examples\LCCC2013\SMC\TrainGateCPS14.xml - UPPAAL

File Edit View Tools Options Help

C:\Users\kgl\Desktop\DESKTOP12\UPPAAL\UPPAAL examples\LCCC2013\SMC\TrainGateCPS14.xml - UPPAAL

File Edit View Tools Options Help

Editor | Simulator | ConcreteSimulator | Verifier | Yggdrasil

- Project
  - Declarations
  - Train
  - Gate
    - Declarations
  - System declarations

Name: Gate Parameters:

```
id_t list[N+1];
int[0,N] len;

// Put an element at the end of the queue
void enqueue(id_t element)
{
        list[len++] = element;
}

// Remove the front element of the queue
void dequeue()
{
        int i = 0;
        len -= 1;
        while (i < len)
        {
                list[i] = list[i + 1];
                i++;
        }
        list[i] = 0;
}

// Returns the front element of the queue
id_t front()
```

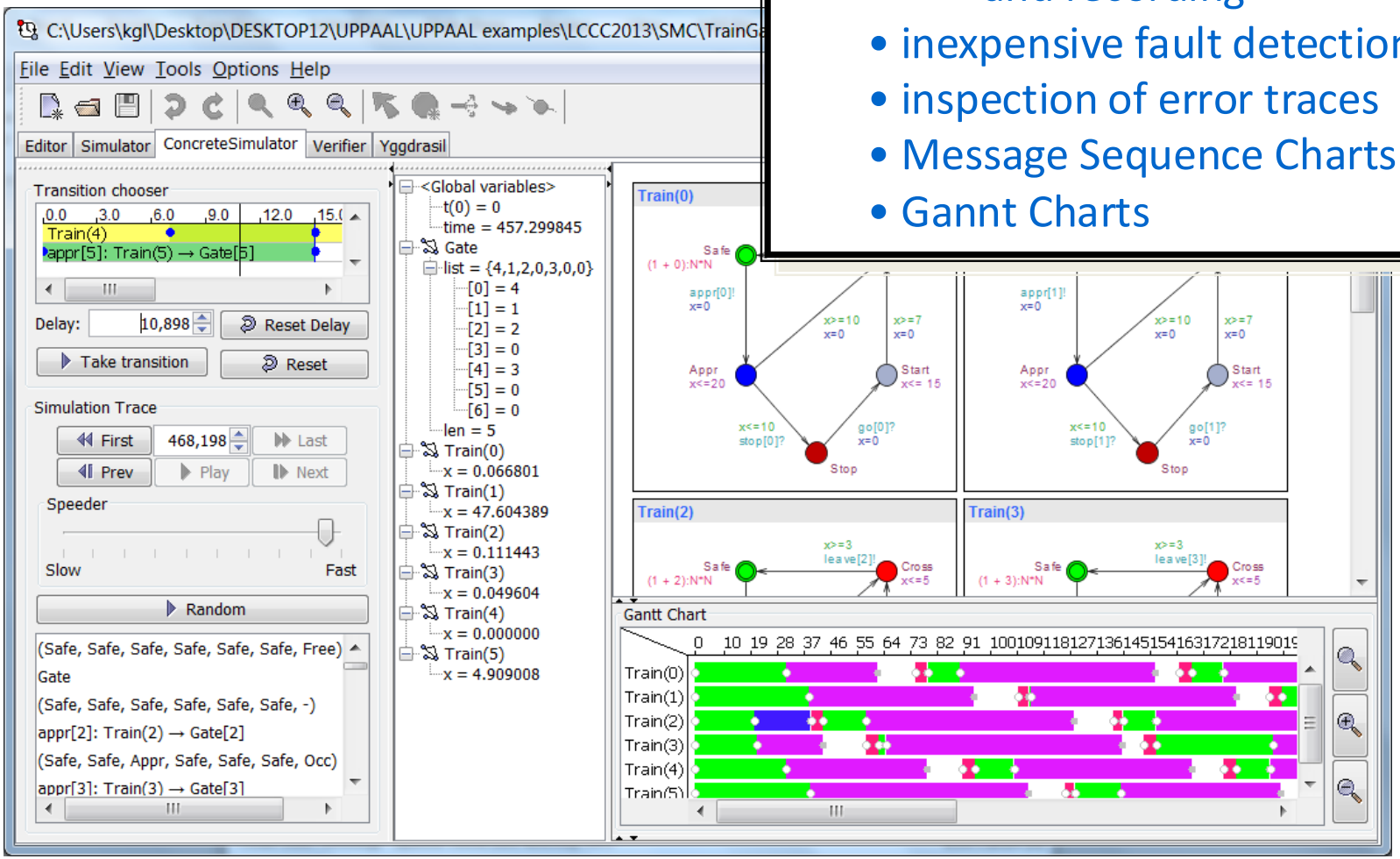
Language

- User defined functions (C-like)
- New types (records, type declarations, meta variables, scalars)
- Partial instantiation of templates
- Select clauses on edges
- Forall and exist quantifiers

Concrete Simulator

Graphical Simulator

- visualization and recording
- inexpensive fault detection
- inspection of error traces
- Message Sequence Charts
- Gannt Charts

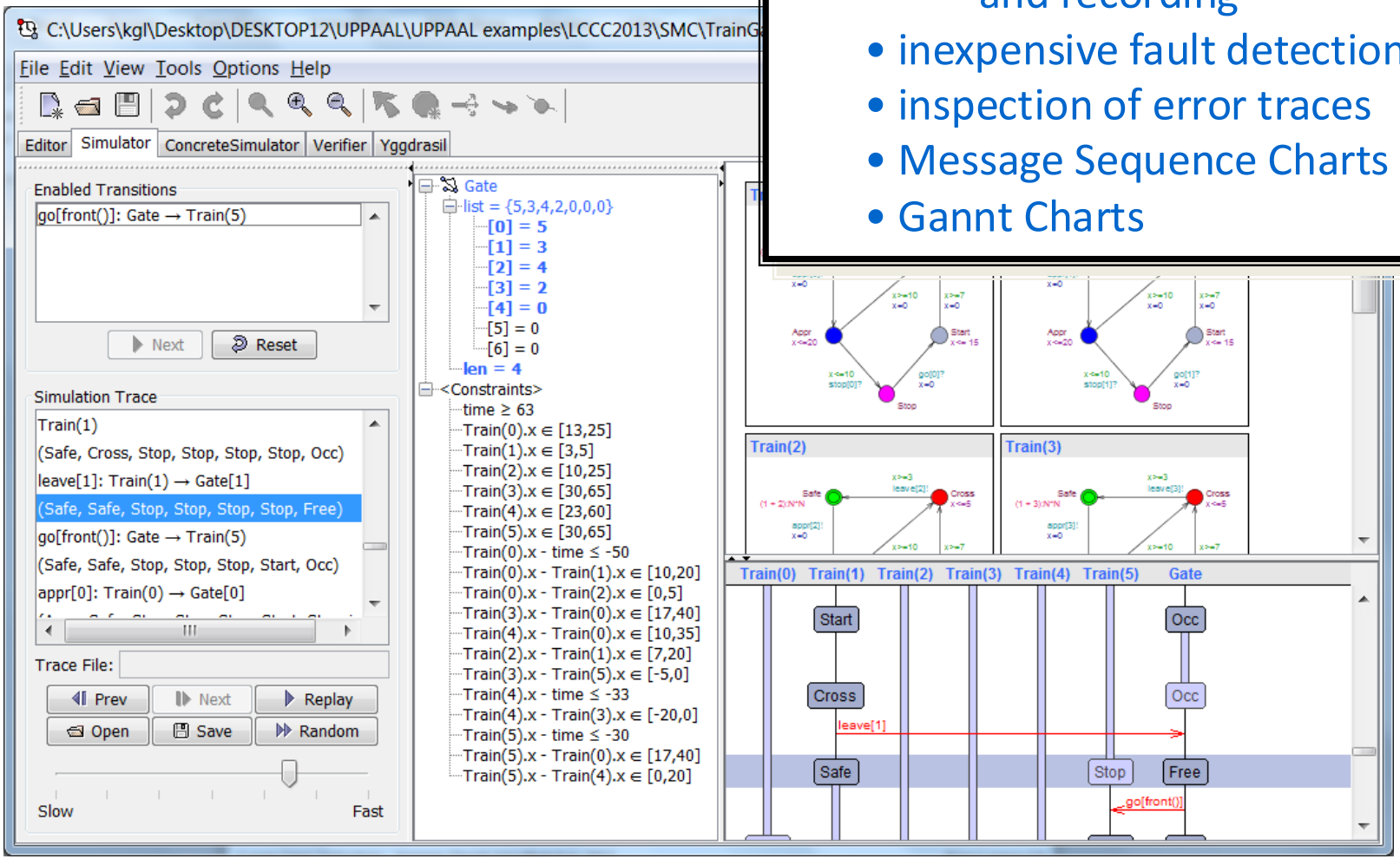


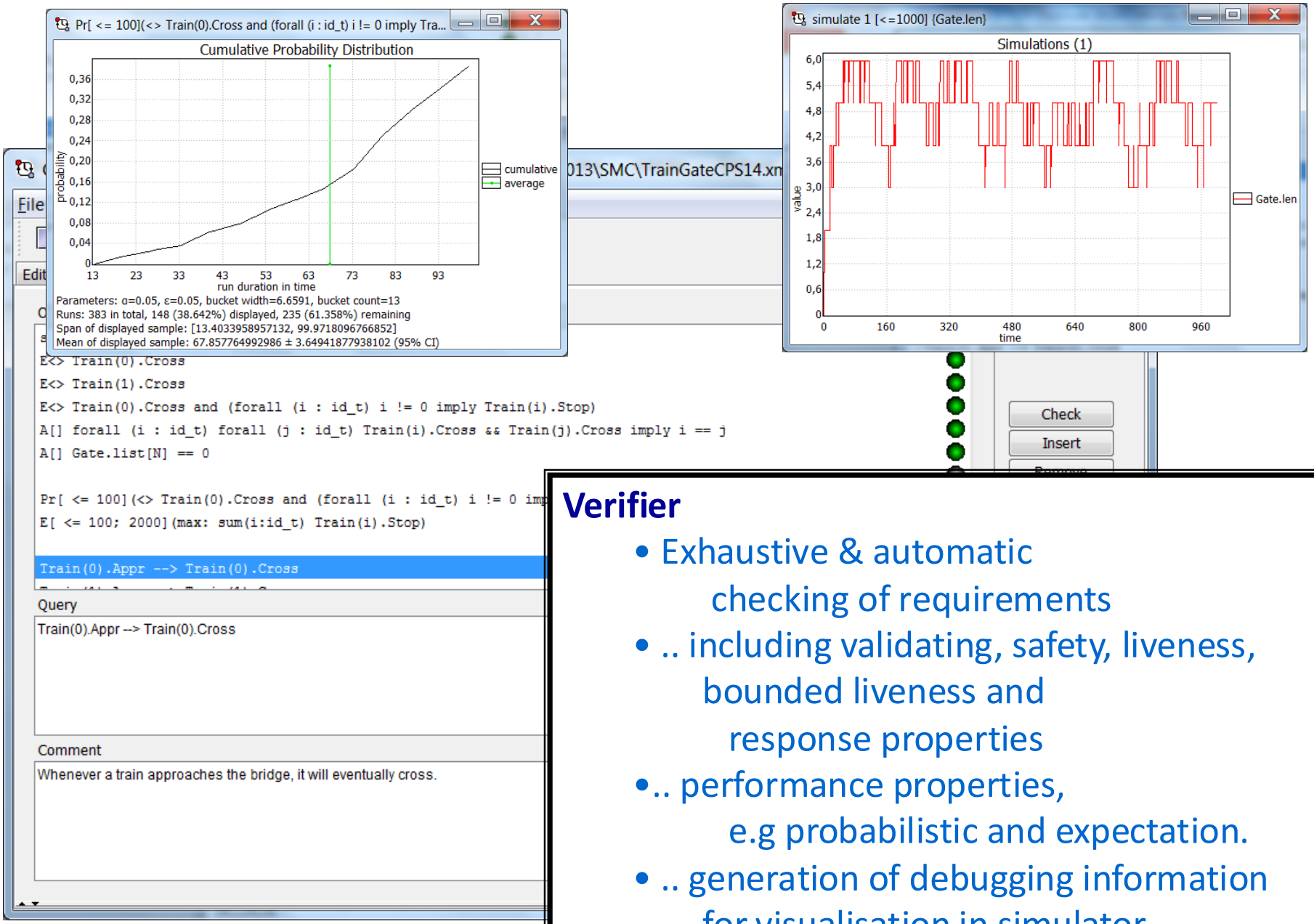
AALBORG UNIVERSITET

Symbolic Simulator

Graphical Simulator

- visualization and recording
- inexpensive fault detection
- inspection of error traces
- Message Sequence Charts
- Gannt Charts





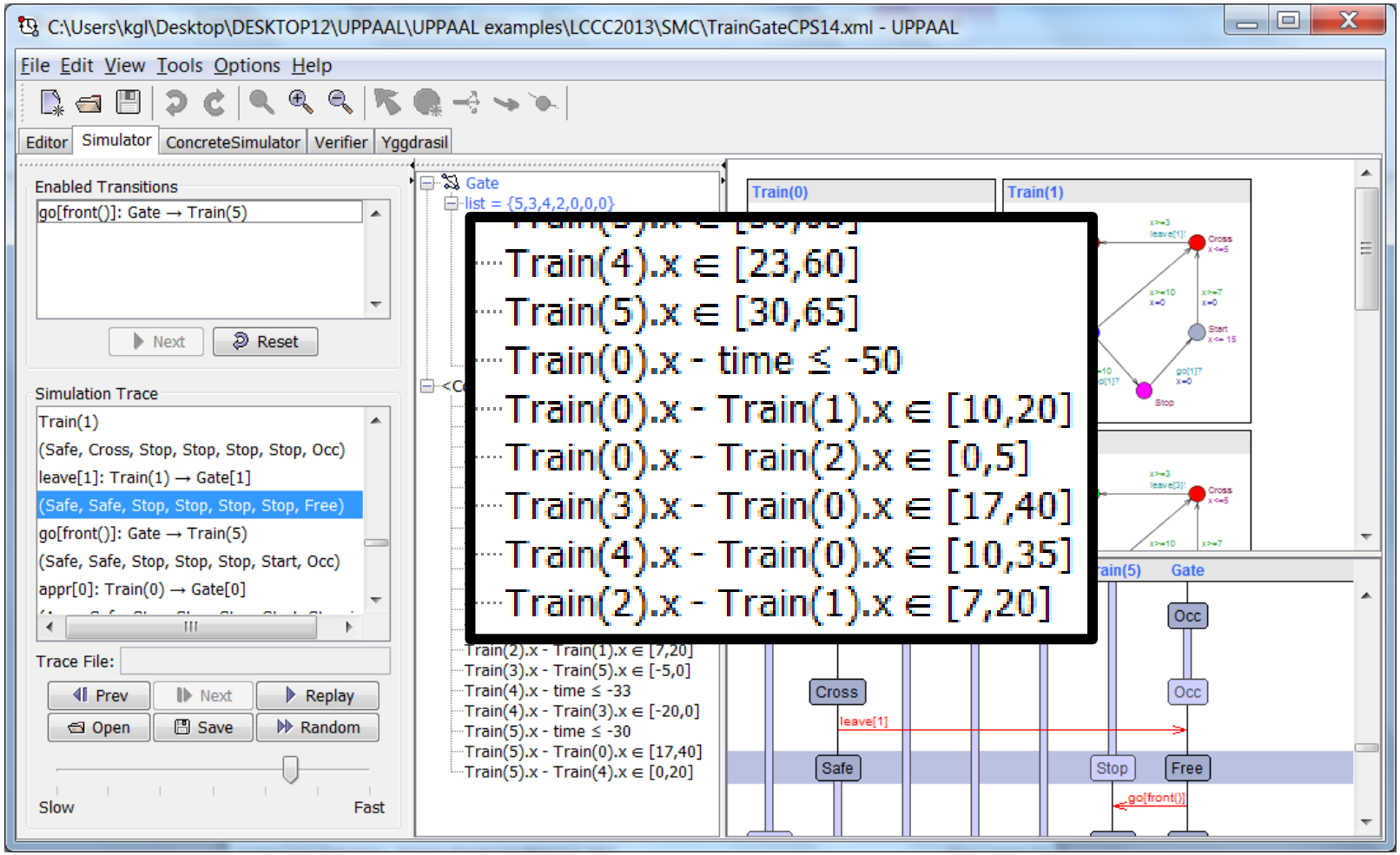
Verifier

- Exhaustive & automatic checking of requirements
- .. including validating, safety, liveness, bounded liveness and response properties
- .. performance properties, e.g probabilistic and expectation.
- .. generation of debugging information for visualisation in simulator.
- .. plot composer

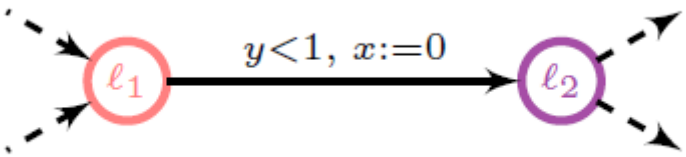
AALBORG UNIVERSITET

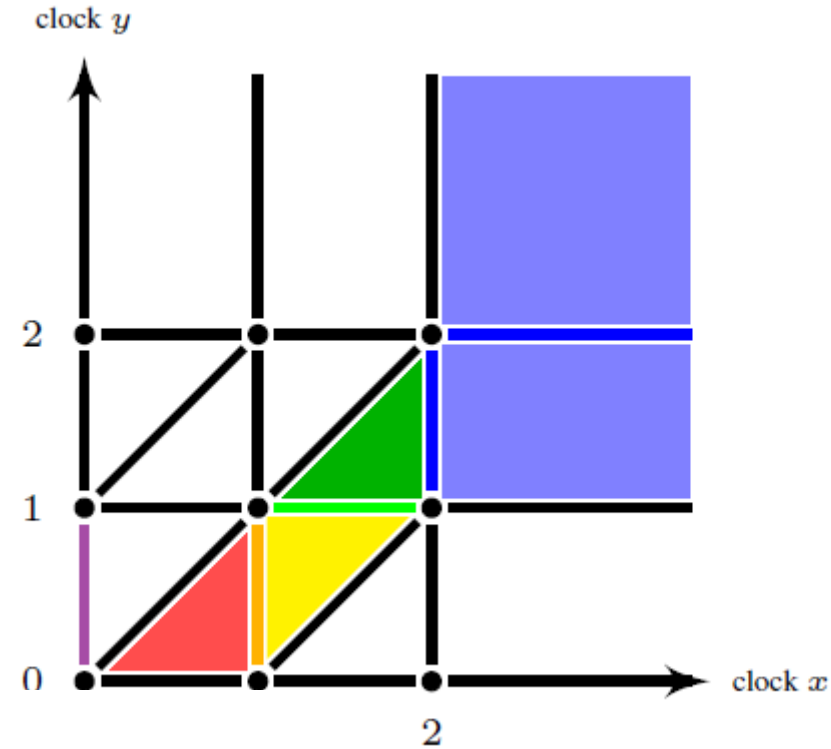
Verification Engine & Options

The ”secret” of UPPAAL



Regions – From Infinite to Finite

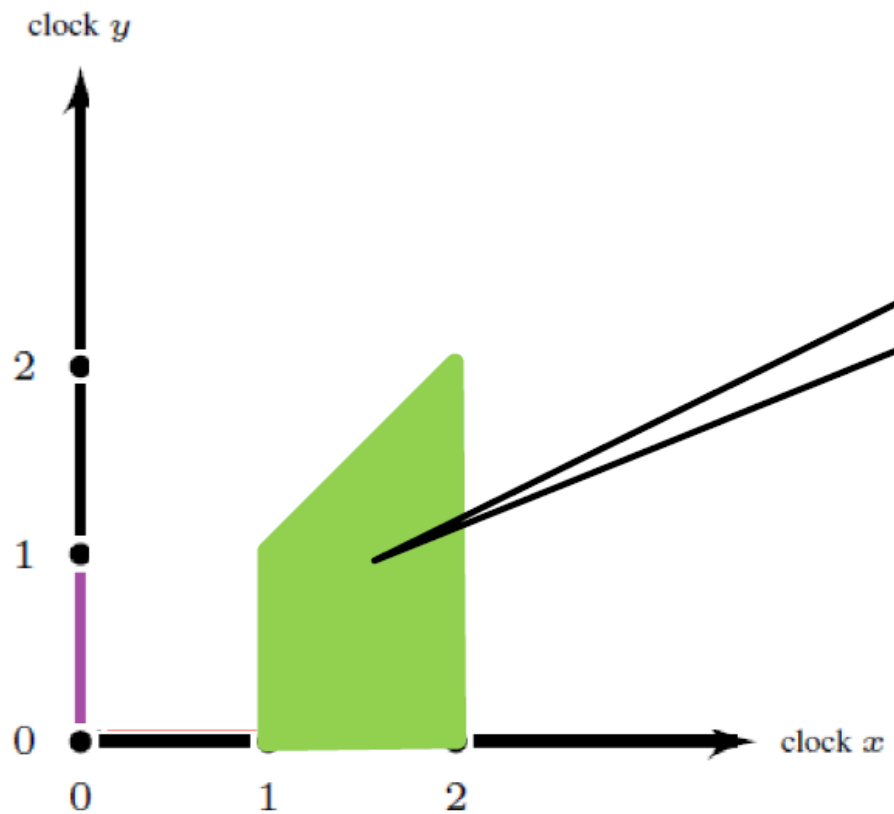




Theorem

The number of regions is $n! \cdot 2^n \cdot \prod_{x \in C}(2c_x + 2)$.

Zones – From Finite to Efficiency



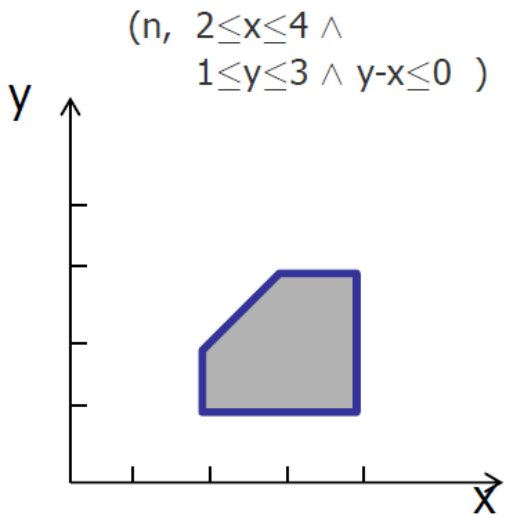
A zone Z:

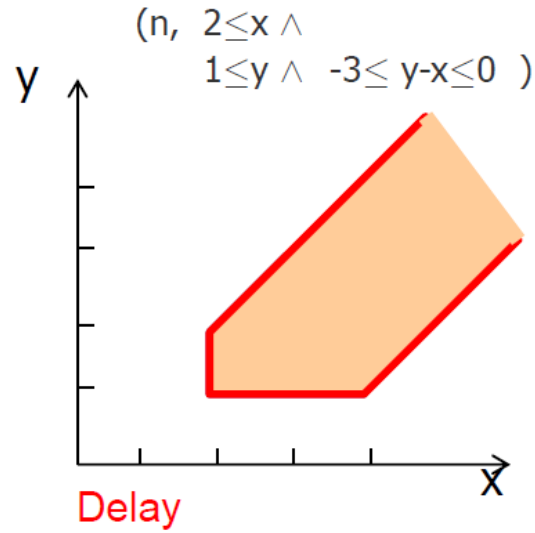
$1 \leq x \leq 2 \quad \wedge$

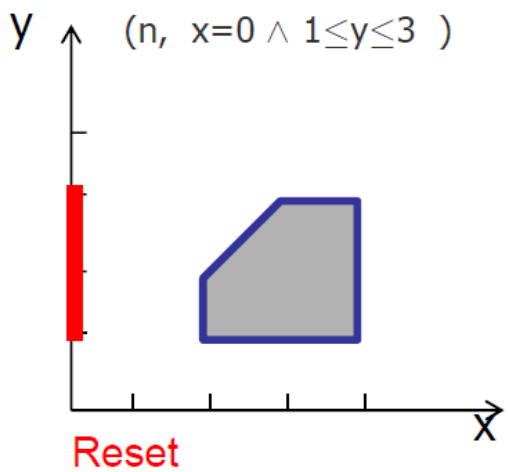
$0 \leq y \leq 2 \quad \wedge$

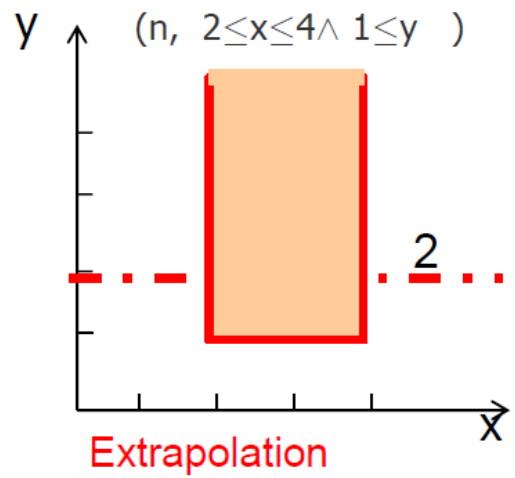
$x - y \geq 0$

Zones - Operations



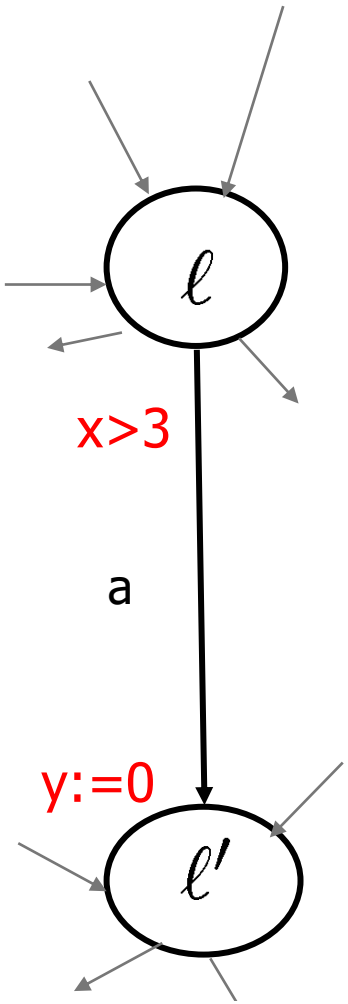


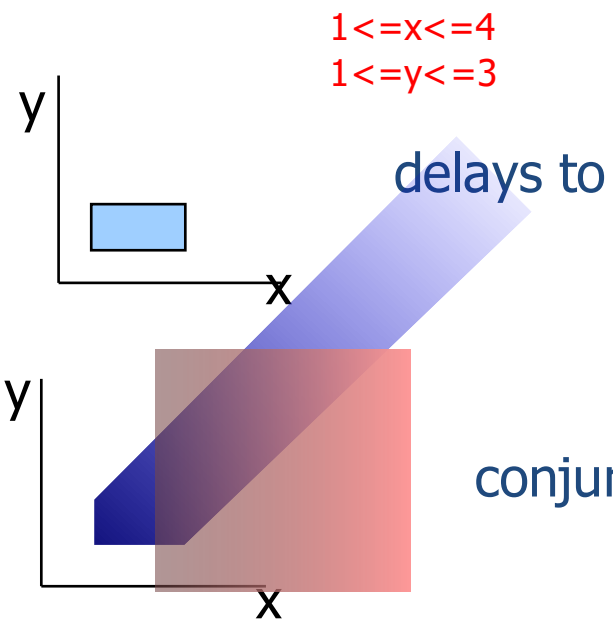




AALBORG UNIVERSITET

Symbolic Transitions

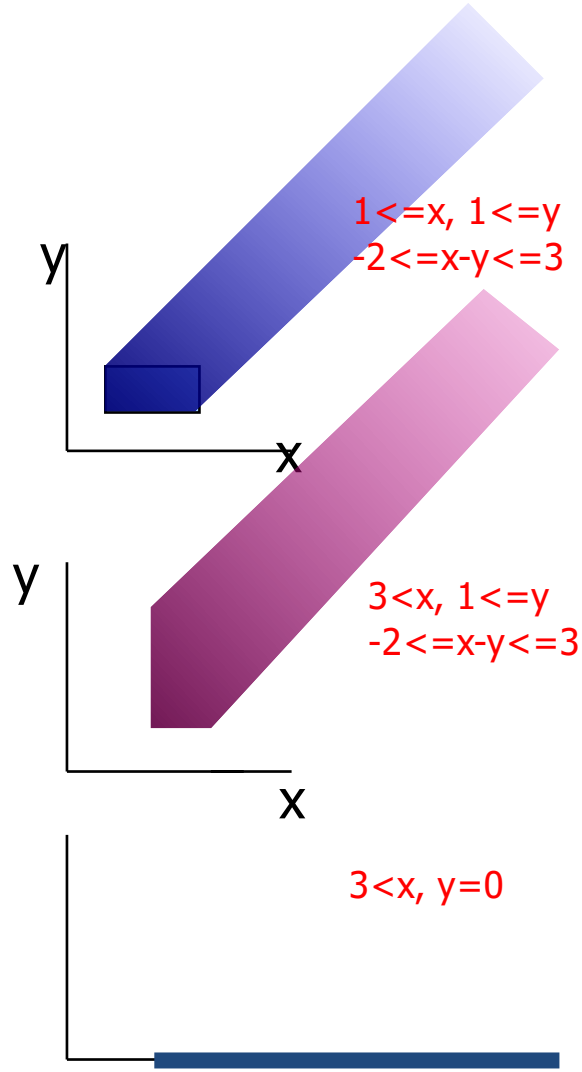




delays to

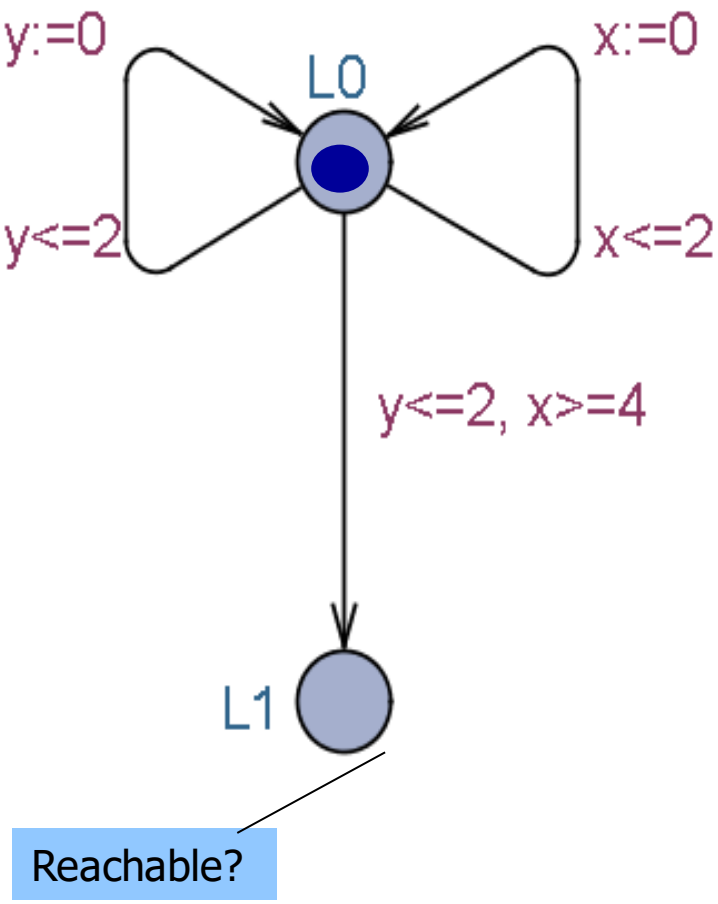
conjuncts to

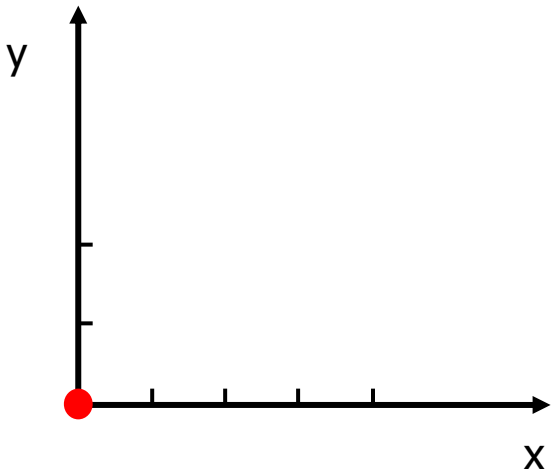
projects to



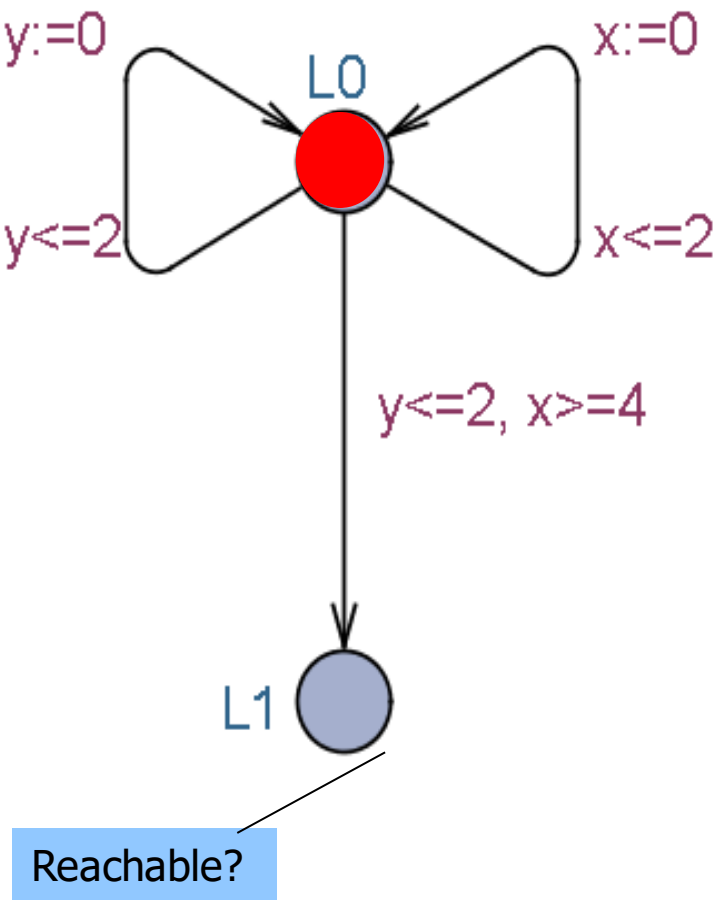
$$(\ell, 1 \leq x \leq 4, 1 \leq y \leq 3) \Rightarrow_a (\ell', 3 < x, y = 0)$$

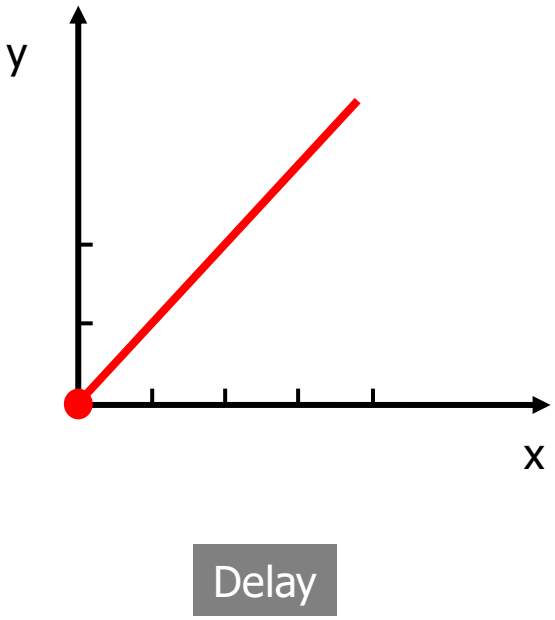
Symbolic Exploration



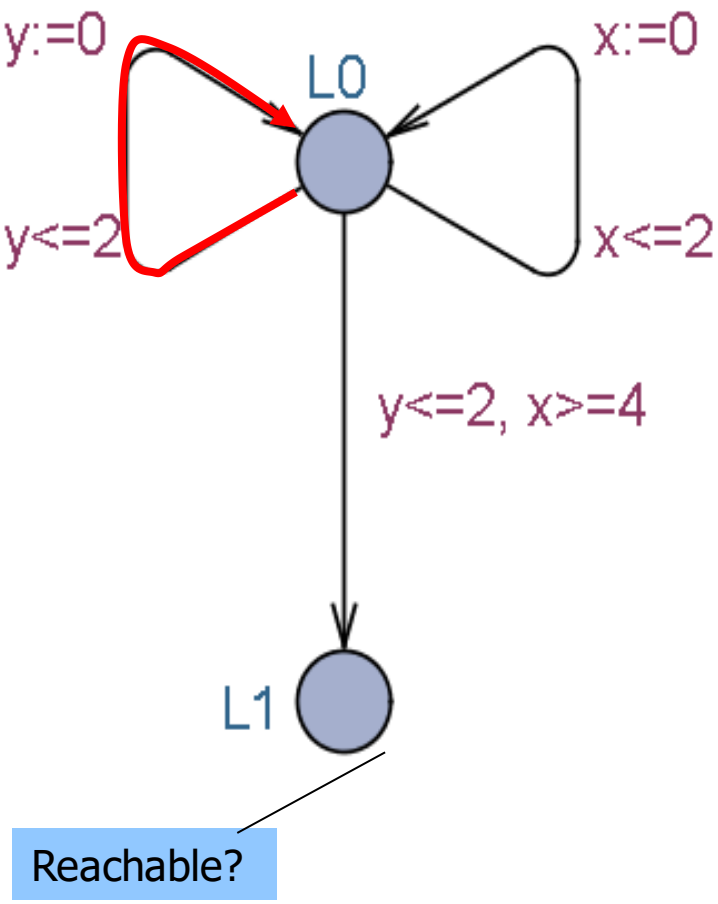


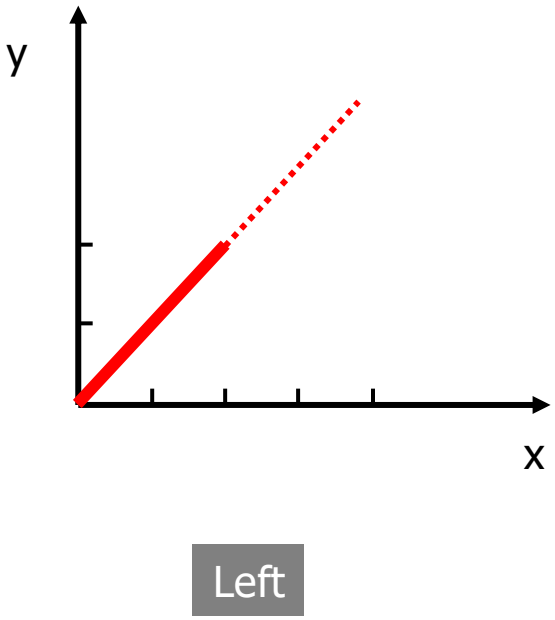
Symbolic Exploration



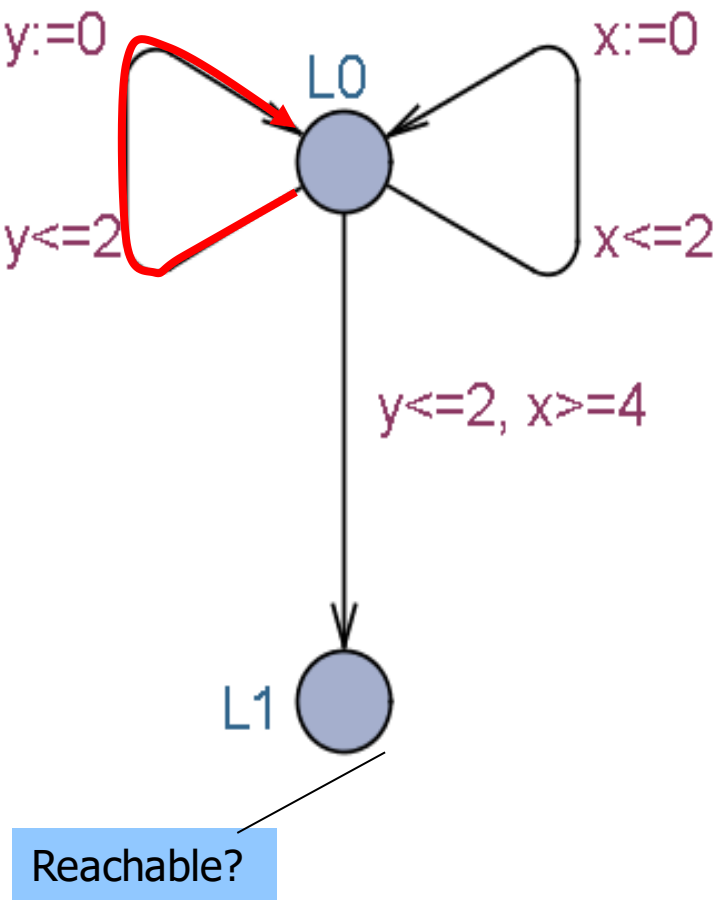


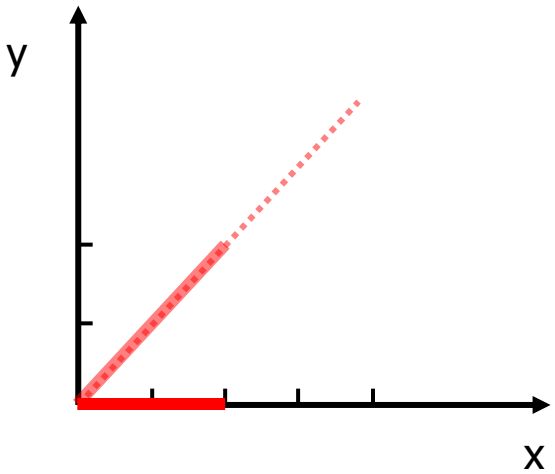
Symbolic Exploration





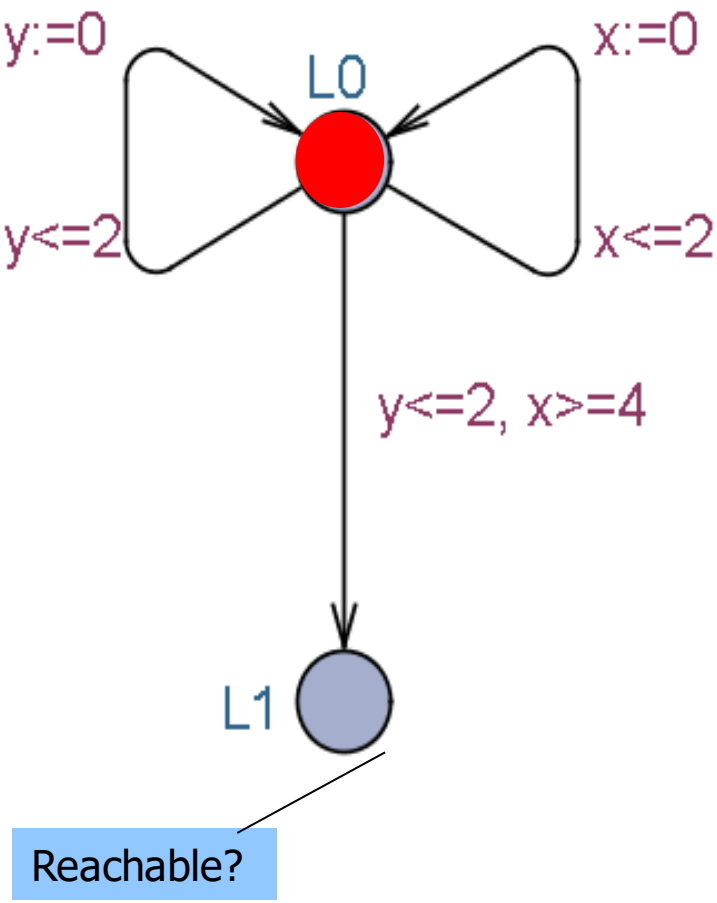
Symbolic Exploration

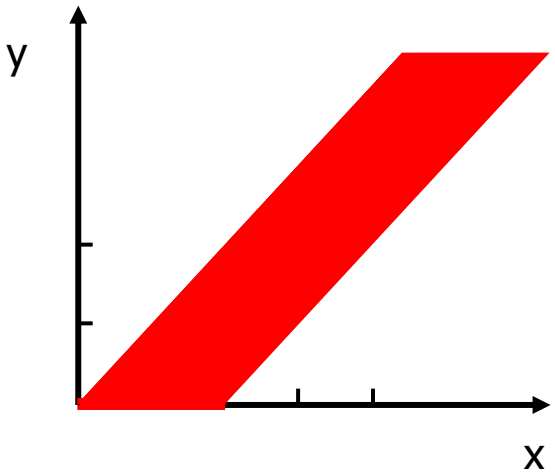




Left

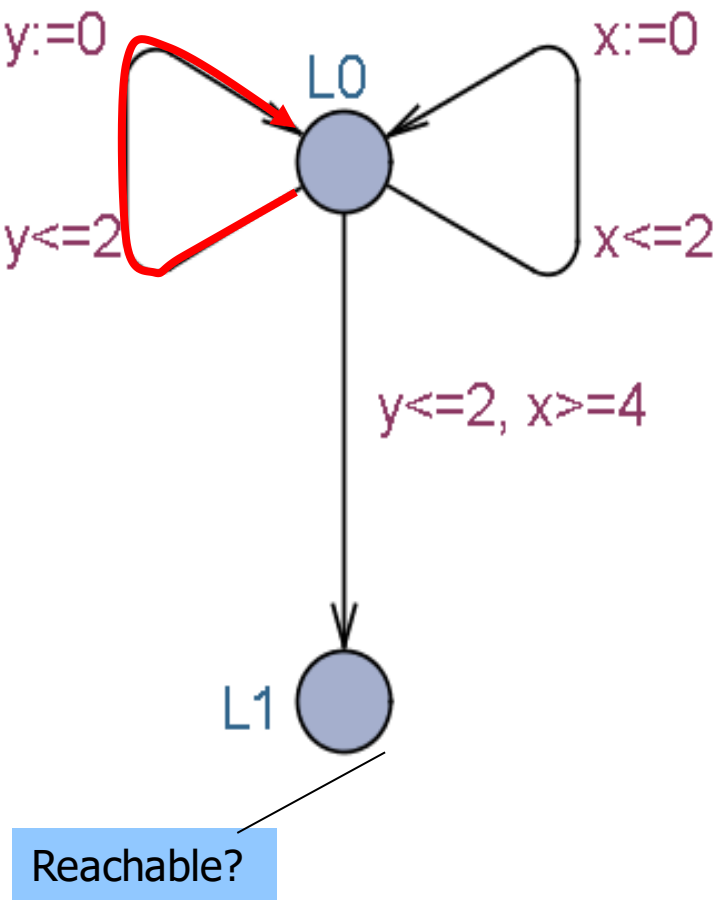
Symbolic Exploration

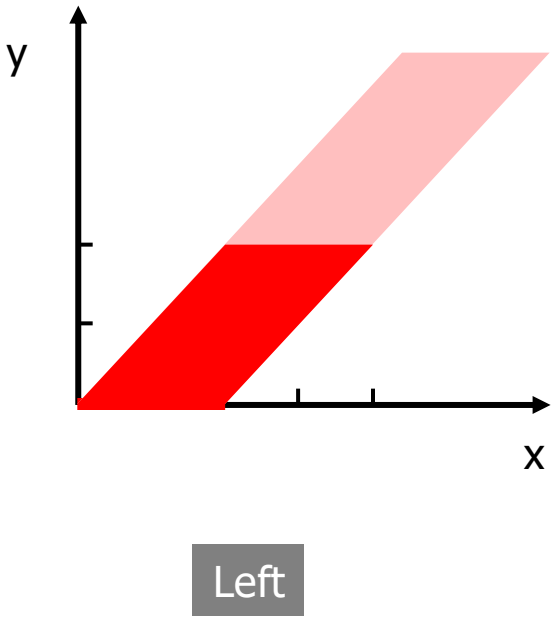




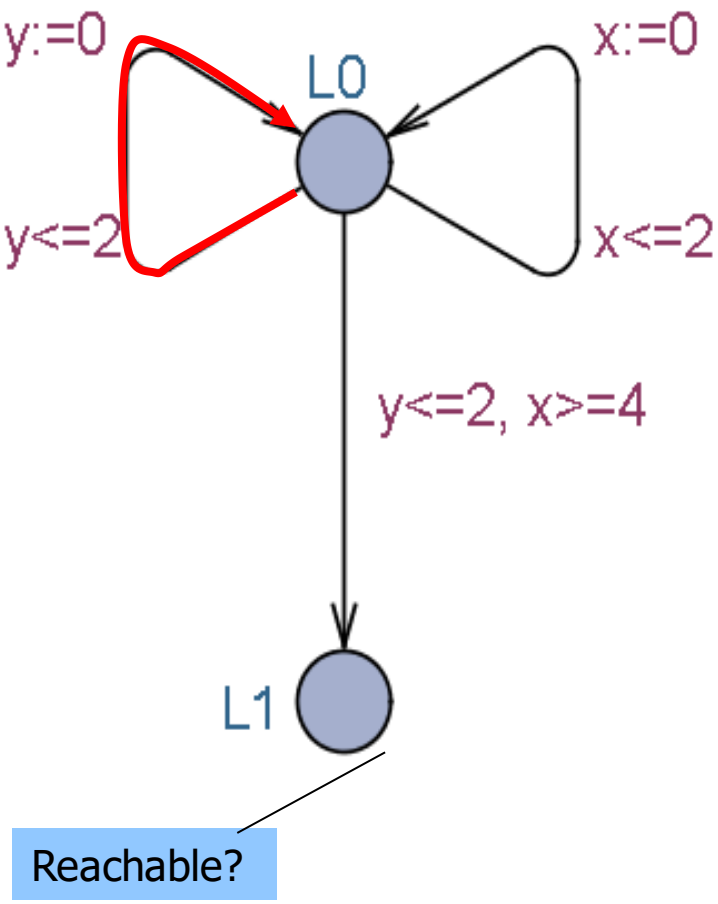
Delay

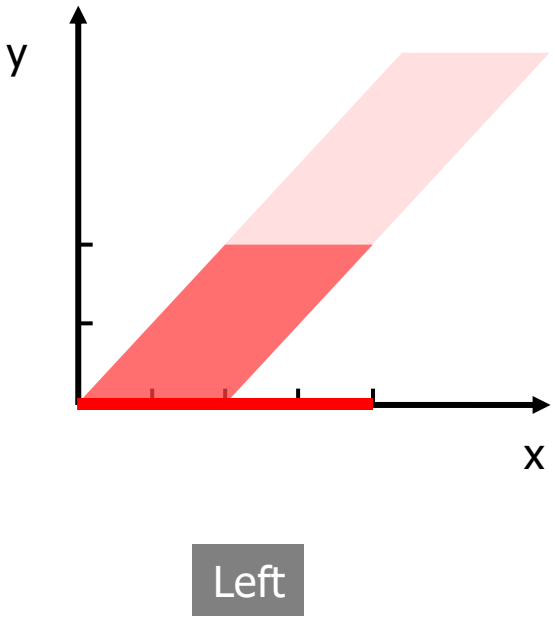
Symbolic Exploration



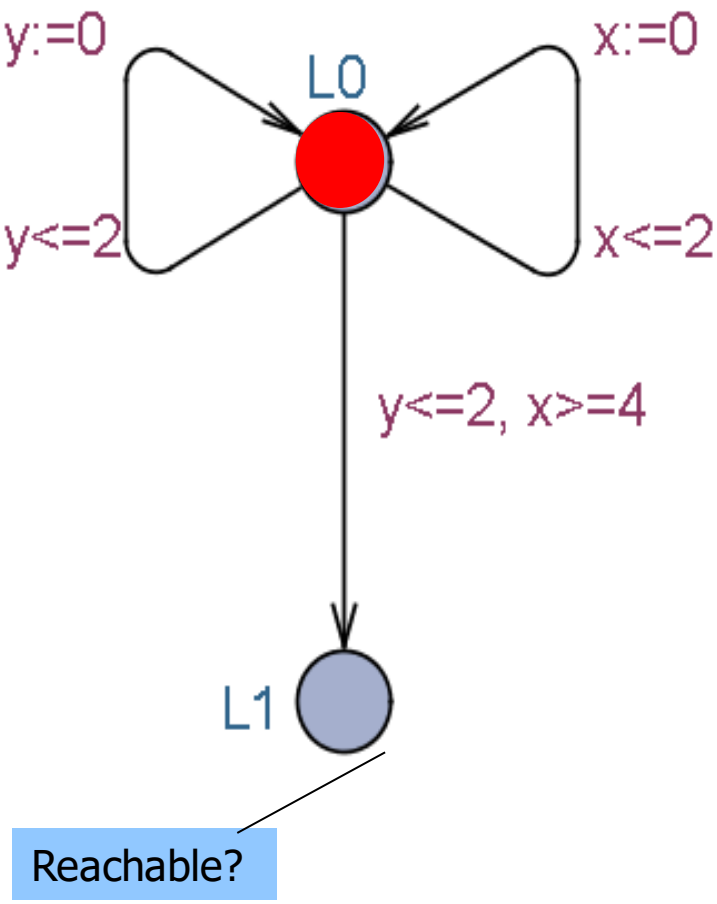


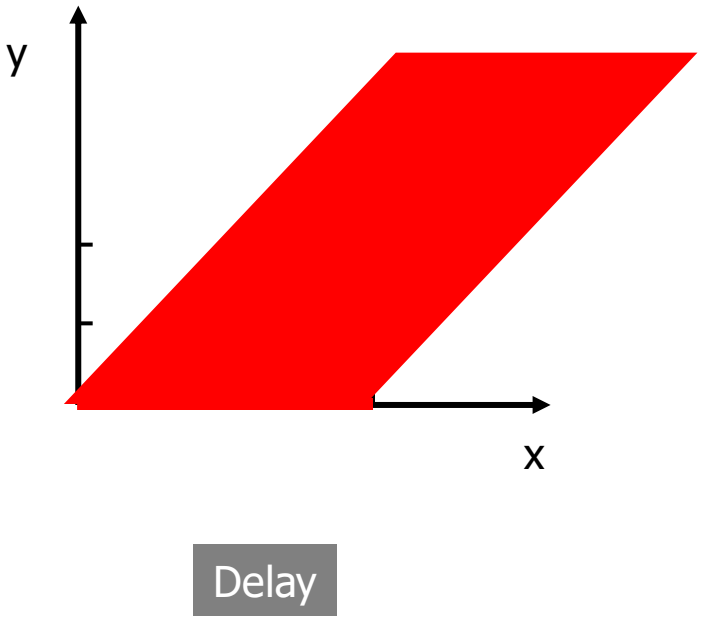
Symbolic Exploration



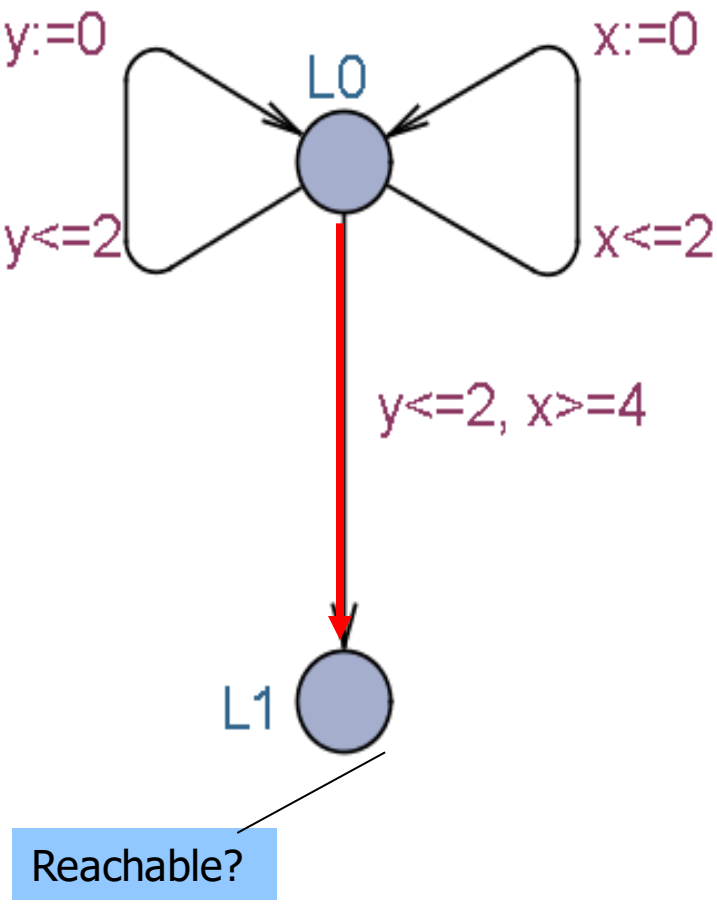


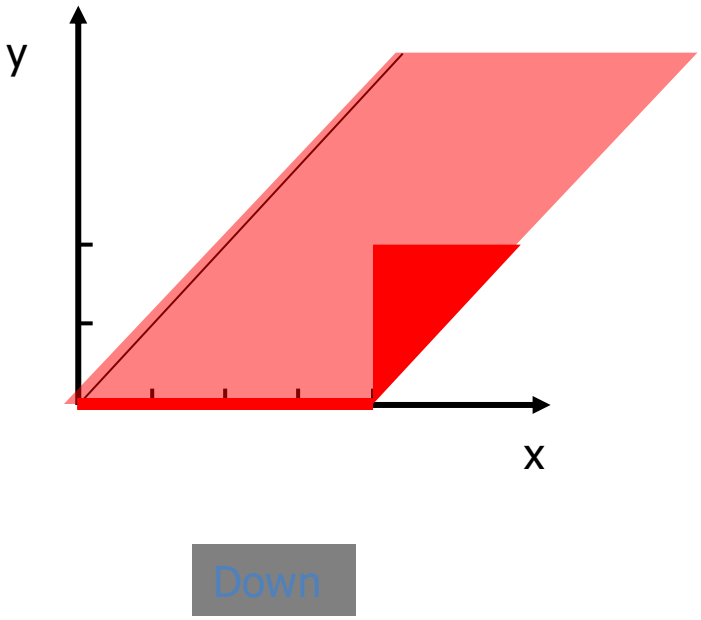
Symbolic Exploration





Symbolic Exploration





Datastructures for Zones

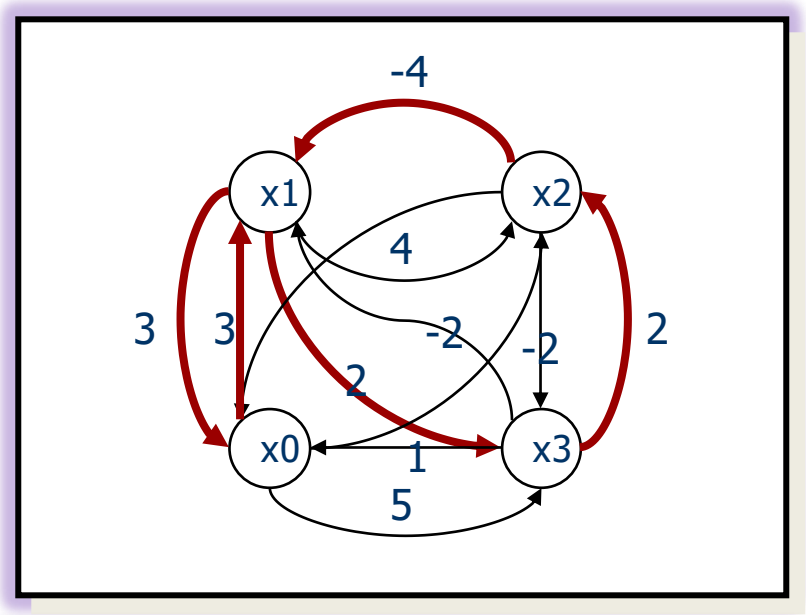
- Difference Bounded Matrices (DBMs)

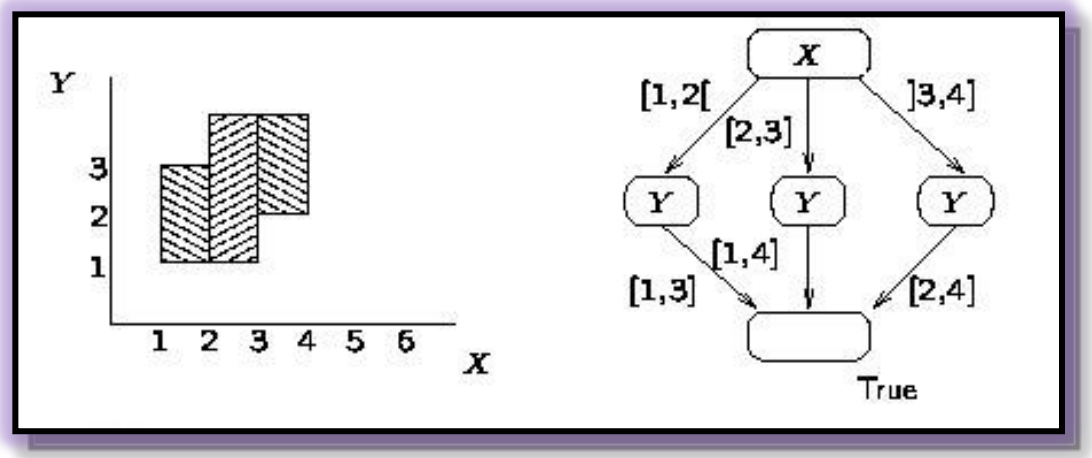
- Minimal Constraint Form

  [RTSS97]

- Clock Difference Diagrams

  [CAV99]





Inclusion Checking (DBMs)

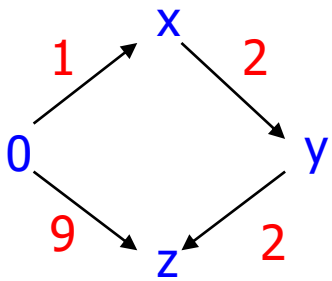
Bellman 1958, Dill 1989

Inclusion

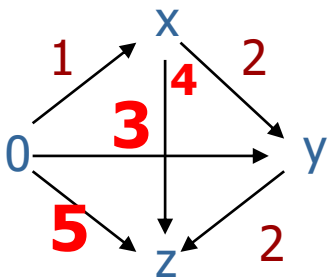
D1

x<=1
y-x<=2
z-y<=2
z<=9

Graph



Shortest Path Closure

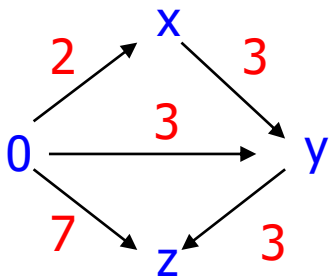


? $\subseteq$ **?**

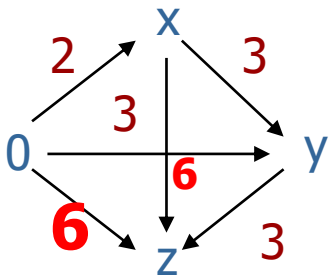
D2

x<=2
y-x<=3
y<=3
z-y<=3
z<=7

Graph

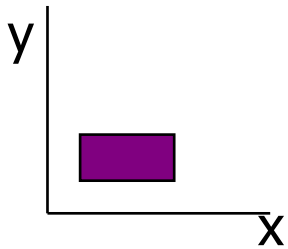


Shortest Path Closure

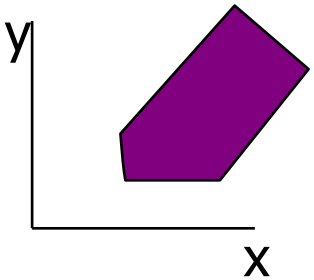


Future (DBMs)

D

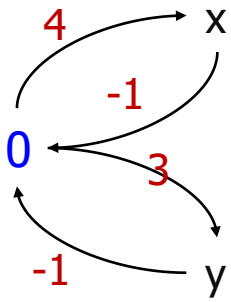


1<= x <=4
1<= y <=3

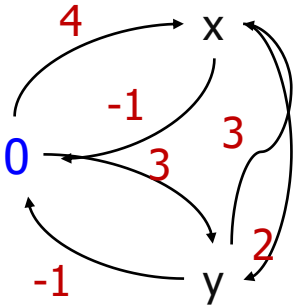


Future D

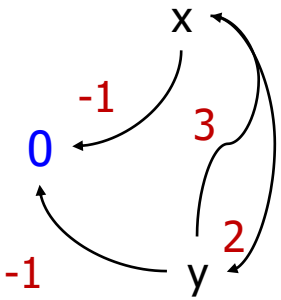
1<=x, 1<=y
-2<=x-y<=3



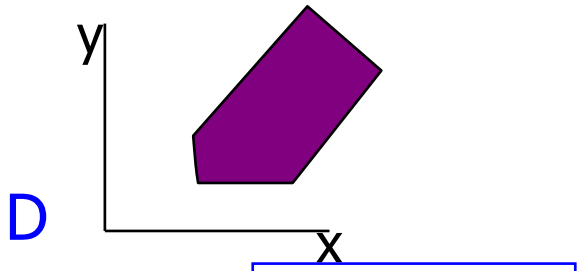
Shortest Path Closure



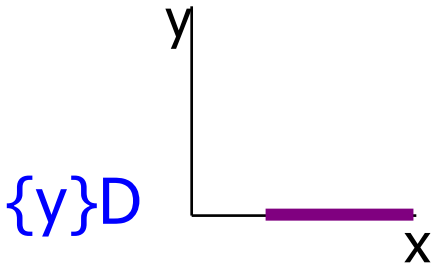
Remove upper bounds on clocks



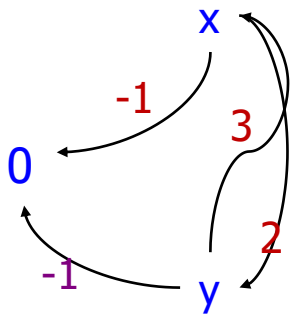
Reset (DBMs)



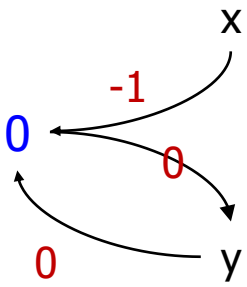
1<=x, 1<=y
-2<=x-y<=3



y=0, 1<=x

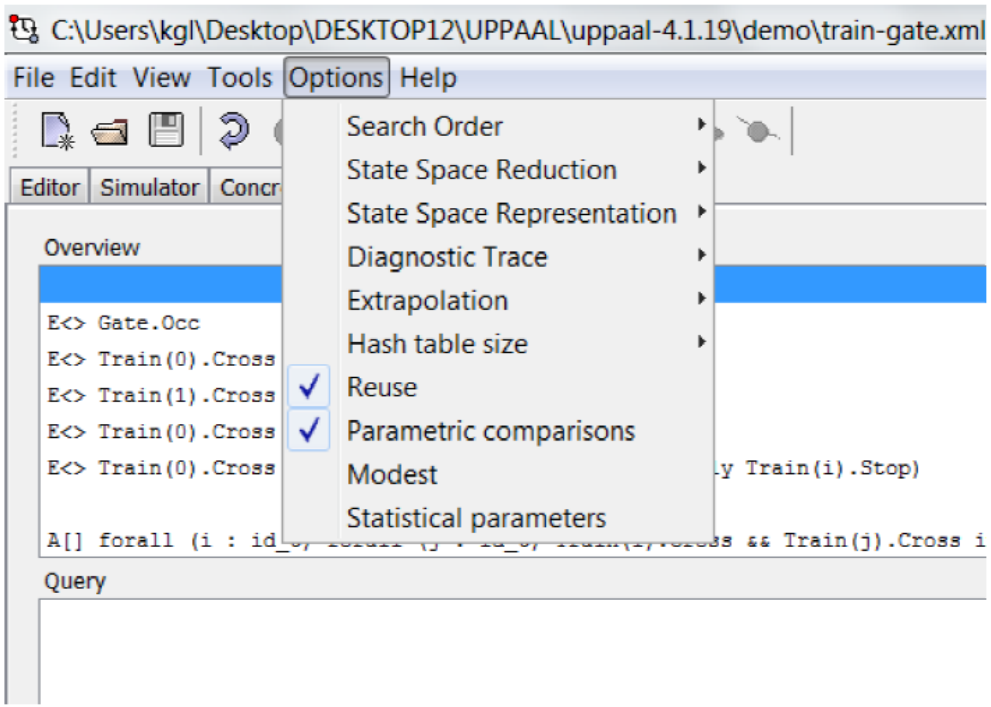


Remove all bounds involving y and set y to 0



Verification Options

Verification Options



Search Order
- Depth First
- Breadth First
- Random Depth First

State Space Reduction
- None
- Conservative
- Aggressive
- Extreme

State Space Representation
- DBM
- Compact Form
- Under Approximation
- Over Approximation

Diagnostic Trace
- Some
- Shortest
- Fastest

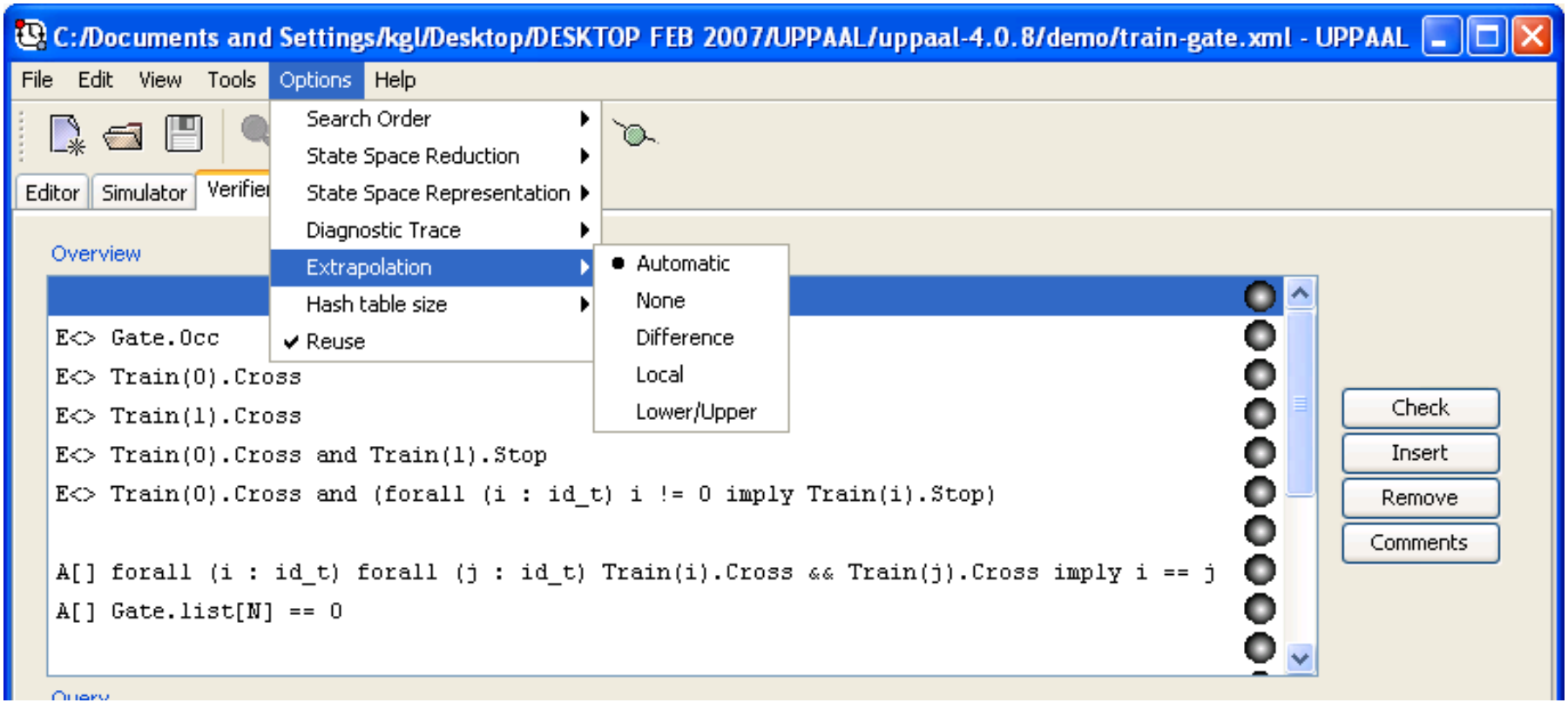
Extrapolation

Hash Table size

Reuse

AALBORG UNIVERSITET

Extrapolation



Tutorials and Video Links

Short Demo: Link

Another Introduction video : Link